

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN BỘ MÔN

CÁC HỆ THỐNG PHÂN TÁN

Đề tài:

**XÂY DỰNG HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ SÁCH
BOOKLAND DỰA TRÊN KIẾN TRÚC MICROSERVICES**

Giảng viên hướng dẫn: Thầy Kim Ngọc Bách

Sinh viên thực hiện: Phạm Thị Minh Anh - B22DCCN041

Trần Xuân Sơn - B22DCCN700

MỤC LỤC

MỤC LỤC.....	1
DANH MỤC HÌNH ẢNH.....	3
DANH MỤC BẢNG BIỂU.....	5
LỜI CẢM ƠN.....	6
LỜI MỞ ĐẦU.....	7
CHƯƠNG 1 - TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT.....	9
1.1 Bối cảnh và lý do chọn đề tài.....	9
1.2 Mô tả bài toán.....	10
1.2 Phạm vi hệ thống.....	11
1.3 Cơ sở lý thuyết.....	12
1.3.1 Kiến trúc Microservice.....	12
1.3.2 Cơ chế giao tiếp đồng bộ qua RESTful API.....	12
1.3.3 Cơ chế giao tiếp bất đồng bộ qua Message Queue - Apache Kafka.....	12
1.4 Bộ cục báo cáo.....	13
CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ HỆ THỐNG.....	14
2.1 Phân tích yêu cầu.....	14
2.1.1 Yêu cầu nghiệp vụ.....	14
2.1.2 Yêu cầu kỹ thuật.....	15
2.1.3 Tiêu chí chất lượng.....	16
2.2 Thiết kế kiến trúc hệ thống.....	16
2.2.1 Kiến trúc tổng quan hệ thống.....	16
2.2.2 Thiết kế các microservice.....	18
2.2.2.1 API Gateway.....	18
2.2.2.2 Identity Service.....	19
2.2.2.3 User Service.....	19
2.2.2.4 Book Service.....	20
2.2.2.5 Order Service.....	20
2.2.2.6 Notification Service.....	21
2.2.2.7 Event Service.....	22
2.2.2.8 File Service.....	22
2.2.2.9 Search Service.....	22
2.2.2.10 Chat Service.....	23
2.2.2.10 Bảng phân rã dịch vụ theo domain-driven design.....	23
2.3 Thiết kế Database - Database per Service.....	24
2.4 Thiết kế RESTful API.....	26
2.4.1 Nguyên tắc thiết kế.....	26

2.4.2 Bảng tổng hợp API Endpoints.....	27
2.4.2.1 Identity Service - /auth/**.....	27
2.4.2.2 User Service — /api/users, /api/addresses, /api/wishlists.....	27
2.4.2.3 Book Service — /api/books, /api/authors, /api/categories.....	28
2.4.2.4 Order Service — /api/carts, /api/bills, /vnpay/**.....	29
2.4.2.5 Notification Service — /api/notifications, /ws/**.....	30
2.4.2.6 Search Service — /api/search/**.....	30
2.4.2.7 Chat Service — /api/chat/**.....	31
2.5 Giao tiếp bất đồng bộ qua Message Queue (Kafka).....	32
2.5.1 Tổng quan Kafka trong hệ thống.....	32
2.5.3 Luồng đặt hàng & Thanh toán.....	33
2.5.4 Luồng đồng bộ Search Index.....	35
2.5.5 Luồng xử lý tin nhắn.....	37
2.5.5 Xử lý lỗi trong Kafka.....	39
2.6 Các tính năng nâng cao.....	39
2.6.1 API Gateway — Spring Cloud Gateway.....	39
2.6.2 Xử lý lỗi cơ bản và định hướng Circuit Breaker.....	40
2.6.3 Redis và định hướng Rate Limiting.....	40
2.6.4 Centralized Logging.....	40
2.6.5 Logging trong từng service.....	42
2.6.6 Observability trên Kubernetes và định hướng Distributed Tracing.....	42
2.6.7 Triển khai trên Kubernetes.....	43
2.6.7.1. Tài nguyên triển khai Kubernetes.....	43
2.6.7.2. Các Nguyên lý Thiết kế Kubernetes Áp dụng trong Dự án.....	44
CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG.....	46
3.1 Triển khai API Gateway.....	46
3.2. Triển khai các microservice.....	46
3.2.1. Triển khai bằng Docker Compose.....	47
3.2.2. Triển khai bằng Kubernetes.....	48
3.3. Triển khai Kafka và giao tiếp bất đồng bộ.....	50
3.4. Triển khai Elasticsearch và Redis lưu trữ file.....	50
3.6. Giao diện frontend.....	51
3.7. Triển khai Deploy.....	55
3.7.1. Deploy FE với Cloudflare Worker.....	55
3.7.2. Deploy BE với Kamatera Cloud và Kubenetes.....	56
3.8. Centralize Logging.....	57
CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	59
4.1 Môi trường thử nghiệm.....	59

4.2 Kiểm thử các nghiệp vụ chính.....	59
4.2.1 Xác thực và phân quyền.....	59
4.2.2 Luồng giỏ hàng, đặt hàng và Kafka notification.....	62
4.2.3 Luồng quản lý sách và tìm kiếm.....	65
4.2.4 Kiểm thử xử lý lỗi cơ bản.....	67
4.2.5 Kiểm thử tính độc lập service.....	70
4.3 Đánh giá kết quả.....	73
4.4 Hạn chế và hướng phát triển.....	73
4.4.1 Hạn chế.....	73
4.4.2 Hướng phát triển.....	74
KẾT LUẬN.....	76
TÀI LIỆU THAM KHẢO.....	78
PHỤ LỤC DỰ ÁN.....	79

DANH MỤC HÌNH ẢNH

Hình 2.1 Sơ đồ kiến trúc tổng quan.....	17
Hình Chat Flow.....	38
Hình 3.1 API Doc Swagger.....	46
Hình 3.2 Các container hệ thống.....	48
Hình 3.3 kubectl get pods.....	49
Hình 3.4 kubectl get services.....	49
Hình 3.5 Trang chủ.....	52
Hình 3.6 Danh sách sản phẩm.....	53
Hình 3.7 Giỏ hàng.....	54
Hình 3.8 Quản lý hóa đơn đã mua.....	55
Hình 3.9 Cloudflare Worker.....	56
Hình 3.10 Kamatera Cloud.....	57
Hình 3.11 Kibana UI.....	58
Hình 4.1 Login.....	60
Hình 4.2 Request GET danh sách ROLE, Header gán Token của tài khoản Admin....	61
Hình 4.3 Request GET danh sách ROLE, không có Token.....	61
Hình 4.4 Request GET danh sách ROLE, Header gán Token của tài khoản User (không có quyền).....	62
Hình 4.5 Checkout hóa đơn.....	63
Hình 4.6 Tạo hóa đơn.....	63
Hình 4.7 Thanh toán.....	64
Hình 4.8 Thanh toán thành công.....	65
Hình 4.9 Hóa đơn đã thanh toán.....	65
Hình 4.10 Tìm kiếm.....	66
Hình 4.11 Dữ liệu trong Elastic DB.....	67
Hình 4.12 Kịch bản Tắt Search Service, Bật Book Service.....	68
Hình 4.13 Kết quả Log.....	69
Hình 4.14 Chuẩn hóa Response.....	69
Hình 4.15 Kịch bản Tắt Search Service, Bật Book Service.....	71
Hình 4.16 Kết quả Log.....	71
Hình 4.17 Kết quả web vẫn hoạt động.....	72

DANH MỤC BẢNG BIỂU

Bảng 2.1 Bảng phân rã dịch vụ theo domain-driven design.....	23
Bảng 2.2 thiết kế database-per-service.....	25
Bảng 2.3 Identity Service API Endpoints.....	27
Bảng 2.4 User Service API Endpoints.....	27
Bảng 2.5 Book Service API Endpoints.....	28
Bảng 2.6 Order Service API Endpoints.....	29
Bảng 2.7 Notification Service API Endpoints.....	30
Bảng 2.8 Search Service API Endpoints.....	31
Bảng 2.9 Chat Service API Endpoints.....	31
Bảng 2.10 Các kafka topic trong hệ thống.....	32

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo và xây dựng hệ thống BookLand, nhóm chúng em đã nhận được sự hướng dẫn, hỗ trợ và tạo điều kiện từ giảng viên, nhà trường cũng như các nguồn tài liệu học thuật liên quan đến lĩnh vực hệ thống phân tán. Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy Kim Ngọc Bách, giảng viên học phần Các hệ thống phân tán, đã truyền đạt những kiến thức nền tảng, cơ sở quan trọng để nhóm có thể phân tích bài toán, lựa chọn kiến trúc phù hợp và triển khai dự án theo đúng định hướng của học phần.

Nhóm chúng em cũng xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông và Khoa Công nghệ Thông tin 1 đã tạo môi trường học tập thuận lợi, giúp sinh viên có điều kiện tiếp cận các kiến thức chuyên ngành và thực hành thông qua các bài tập lớn có tính ứng dụng. Trong quá trình thực hiện đồ án, nhóm đã có cơ hội củng cố kiến thức đồng thời hiểu rõ hơn những khó khăn thực tế khi thiết kế một hệ thống gồm nhiều service độc lập.

Do thời gian thực hiện và kinh nghiệm triển khai hệ thống phân tán còn hạn chế, báo cáo và sản phẩm không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý của thầy để có thể tiếp tục hoàn thiện hệ thống, bổ sung các cơ chế nâng cao và rút kinh nghiệm cho các dự án sau.

LỜI MỞ ĐẦU

Trong bối cảnh các ứng dụng phần mềm ngày càng phải phục vụ số lượng người dùng lớn, xử lý nhiều luồng nghiệp vụ đồng thời và đáp ứng yêu cầu mở rộng linh hoạt, kiến trúc hệ thống phân tán đã trở thành một hướng tiếp cận quan trọng trong phát triển phần mềm hiện đại. Thay vì xây dựng toàn bộ ứng dụng trong một khối nguyên khối duy nhất, kiến trúc microservice cho phép chia hệ thống thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một miền nghiệp vụ riêng và có thể được phát triển, triển khai, mở rộng một cách tương đối độc lập.

Đề tài “*Xây dựng hệ thống thương mại điện tử sách BookLand dựa trên kiến trúc Microservices*” được thực hiện nhằm vận dụng các kiến thức đã học trong môn Các hệ thống phân tán vào một bài toán có tính thực tiễn. Hệ thống BookLand mô phỏng một nền tảng bán sách trực tuyến, trong đó người dùng có thể đăng ký, đăng nhập, tìm kiếm sách, xem chi tiết sản phẩm, quản lý giỏ hàng, đặt hàng, thanh toán và nhận thông báo. Bên cạnh các chức năng dành cho khách hàng, hệ thống còn cung cấp các chức năng quản trị như quản lý sách, tác giả, danh mục, sự kiện, phương thức vận chuyển và đơn hàng.

Về mặt kỹ thuật, hệ thống được thiết kế theo hướng tách các chức năng chính thành nhiều microservice như Identity Service, User Service, Book Service, Order Service, Event Service, Notification Service, File Service và Search Service. Các service giao tiếp với nhau thông qua RESTful API đối với các tác vụ cần phản hồi trực tiếp và thông qua Apache Kafka đối với các luồng xử lý bất đồng bộ như gửi thông báo. Ngoài ra, hệ thống còn sử dụng các công nghệ hỗ trợ như Docker Compose, MySQL, MongoDB, Elasticsearch, Redis, MinIO và WebSocket để mô phỏng một môi trường triển khai gần với thực tế.

Báo cáo này trình bày toàn bộ quá trình thực hiện đề tài, bao gồm tổng quan bài toán, cơ sở lý thuyết, phân tích yêu cầu, thiết kế kiến trúc, thiết kế cơ sở dữ liệu, thiết kế API, giao tiếp bất đồng bộ, triển khai hệ thống, kiểm thử và đánh giá kết quả. Thông qua đề tài, nhóm mong muốn làm rõ cách một hệ thống thương mại điện tử có

thể được tổ chức theo kiến trúc microservice, đồng thời chỉ ra những ưu điểm, hạn chế và hướng phát triển tiếp theo của hệ thống.

CHƯƠNG 1 - TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh nền kinh tế số phát triển vượt bậc, thương mại điện tử đã trở thành một phần không thể thiếu trong thói quen tiêu dùng toàn cầu nói chung và tại Việt Nam nói riêng. Trong số các ngành hàng trực tuyến, kinh doanh sách và các sản phẩm văn hóa số luôn chiếm một tỷ trọng lớn nhờ vào sự phổ cập của văn hóa đọc và các giải pháp học tập trực tuyến. Các hệ thống thương mại điện tử hiện đại ngày nay không chỉ đơn thuần là nơi trưng bày sản phẩm, mà đã dịch chuyển thành các hệ sinh thái phức hợp, tích hợp từ quản lý định danh, tìm kiếm thông minh, xử lý đơn hàng thời gian thực, cho đến các hệ thống phân tích hành vi và gửi thông báo tự động.

Sự bùng nổ về số lượng người dùng và lưu lượng truy cập cực đại vào các khung giờ cao điểm (giờ vàng khuyến mãi, sự kiện ra mắt sách) đặt ra một thách thức rất lớn đối với kiến trúc hạ tầng phần mềm. Kiến trúc đơn khối truyền thống - nơi toàn bộ logic ứng dụng và cơ sở dữ liệu được đóng gói trong một tiến trình duy nhất dần bộc lộ những hạn chế chí mạng: Khả năng co giãn kém - khi một phân hệ bị quá tải toàn bộ hệ thống sẽ bị chậm hoặc sụp đổ theo, không thể mở rộng tài nguyên riêng biệt cho phân hệ đó; rủi ro sụp đổ dây chuyền (Single Point of Failure) - một lỗi nhỏ phát sinh tại dịch vụ gửi email hay quản lý tệp tin có thể kéo sụp toàn bộ ứng dụng, làm gián đoạn trải nghiệm mua sắm của khách hàng; ràng buộc về công nghệ - đội ngũ phát triển bị bó buộc vào một ngôn ngữ lập trình và một hệ quản trị cơ sở dữ liệu duy nhất cho toàn bộ hệ thống.

Để giải quyết triệt để các bài toán này, kiến trúc Microservices kết hợp với mô hình Hướng sự kiện đã trở thành xu hướng công nghệ tiên quyết được các tập đoàn công nghệ lớn áp dụng. Bằng cách phân rã ứng dụng thành các dịch vụ nhỏ, độc lập về cả logic lẫn tầng lưu trữ dữ liệu, hệ thống có thể đạt được tính sẵn sàng cao, khả năng cô lập lỗi tối đa và linh hoạt trong việc co giãn theo tải thực tế.

Tuy nhiên, việc chuyển dịch sang môi trường phân tán cũng làm phát sinh các bài toán thách thức mang tính kinh điển của môn học Hệ phân tán, bao gồm: cơ chế

định tuyến qua API Gateway, giao tiếp liên dịch vụ (IPC), đảm bảo tính nhất quán cuối cùng của dữ liệu và năng lực chịu lỗi cục bộ khi các dịch vụ tạm thời mất kết nối.

Nhận thức được tầm quan trọng của các nguyên lý thiết kế hệ thống phân tán trong thực tiễn, nhóm nghiên cứu đã quyết định lựa chọn đề tài “*Xây dựng hệ thống thương mại điện tử sách BookLand dựa trên kiến trúc Microservices*”. Đề tài này không chỉ dừng lại ở việc hiện thực hóa một ứng dụng thương mại điện tử hoàn chỉnh phục vụ người dùng, mà quan trọng hơn, là môi trường thực nghiệm để giải quyết các bài toán cốt lõi của hệ phân tán: sử dụng RESTful API cho giao tiếp đồng bộ, tối ưu thông lượng bằng Apache Kafka cho giao tiếp bất đồng bộ, đồng thời triển khai các giải pháp chịu lỗi và giám sát hệ thống nâng cao.

1.2 Mô tả bài toán

Dự án BookLand là một hệ thống thương mại điện tử chuyên về kinh doanh sách trực tuyến, được xây dựng theo kiến trúc microservice với đầy đủ các nghiệp vụ thương mại điện tử. Về mặt chức năng, hệ thống cho phép người dùng đăng ký/đăng nhập, tìm kiếm sách, xem thông tin chi tiết sản phẩm, quản lý giỏ hàng, đặt hàng, thanh toán, nhận thông báo và theo dõi các chương trình khuyến mãi. Bên cạnh đó, hệ thống cũng cung cấp các chức năng quản trị như quản lý sách, tác giả, danh mục, người dùng, đơn hàng, sự kiện và phương thức vận chuyển.

Về mặt kiến trúc và vận hành, yêu cầu đặt ra là hệ thống phải được tách thành nhiều service độc lập. Mỗi dịch vụ cần sở hữu vùng lưu trữ dữ liệu độc lập để tránh phụ thuộc chéo. Để hệ thống có thể vận hành hiệu quả, linh hoạt và đáp ứng khả năng mở rộng, bài toán đặt ra yêu cầu phải giải quyết được 4 thách thức kỹ thuật cốt lõi sau:

1. Phân rã dịch vụ và Quản lý dữ liệu độc lập: hệ thống phải được chia nhỏ thành các vi dịch vụ hoạt động độc lập (như User Service, Book Service, Order Service...). Mỗi dịch vụ phải quản lý một cơ sở dữ liệu riêng biệt để đảm bảo tính độc lập tuyệt đối, dẫn đến bài toán làm sao để duy trì tính nhất quán của dữ liệu trên toàn hệ thống khi một nghiệp vụ bán hàng liên quan đến nhiều bảng dữ liệu nằm ở các máy chủ khác nhau.

2. Giao tiếp liên dịch vụ: Kết hợp linh hoạt giữa giao tiếp đồng bộ (để truy vấn dữ liệu tức thì như thông tin sách, giá cả) và giao tiếp bất đồng bộ (để xử lý các luồng công việc chạy ngầm như tạo đơn hàng, gửi thông báo, cập nhật chỉ mục tìm kiếm nhằm giảm tải cho hệ thống)
3. Kiểm soát và xử lý lỗi cơ bản: Trong môi trường phân tán, hệ thống bắt buộc phải có phương án xử lý khi xảy ra tình huống một dịch vụ tạm thời mất kết nối hoặc thông điệp giao tiếp bị lỗi, đảm bảo lỗi của một phân hệ không làm sụp đổ dây chuyền toàn bộ ứng dụng
4. Giám sát hệ thống: Cơ chế ghi vết hoạt động (logging) ở các mức độ khác nhau (thông tin, cảnh báo, lỗi) để người quản trị có thể theo dõi luồng đi của dữ liệu và định vị sự cố kịp thời.

Như vậy, dự án BookLand không chỉ tập trung vào việc xây dựng các chức năng bán sách trực tuyến, mà còn hướng đến việc minh họa các đặc trưng quan trọng của hệ thống phân tán. Các đặc trưng này bao gồm phân tách dịch vụ, giao tiếp liên dịch vụ, xử lý bất đồng bộ, quản lý dữ liệu độc lập, nâng cao năng lực chịu lỗi cục bộ triển khai bằng container và khả năng mở rộng từng thành phần.

1.2 Phạm vi hệ thống

Hệ thống phần mềm này được tổ chức theo kiến trúc microservice với 10 dịch vụ độc lập, bao gồm API Gateway, Identity, User, Book, Order, Notification, Event, File, Search và Chat. Mỗi dịch vụ đảm nhận một phạm vi chức năng riêng biệt và giao tiếp qua các API nội bộ, tạo điều kiện cho việc mở rộng, bảo trì và triển khai độc lập.

Phần giao diện người dùng được xây dựng bằng React kết hợp Vite và TypeScript. Tuy nhiên, thành phần frontend này tách biệt với lõi backend của dự án và không nằm trong phạm vi trọng tâm của kiến trúc microservice hiện tại.

Về hạ tầng, dự án hỗ trợ cả môi trường chạy container qua Docker Compose và triển khai trên Kubernetes (Minikube). Hệ thống cũng tích hợp các thành phần cơ sở hạ tầng quan trọng như MinIO cho lưu trữ đối tượng, Elasticsearch cho tìm kiếm,

Kafka cho xử lý bất đồng bộ và truyền thông tin, Redis cho caching, cùng MySQL và MongoDB làm cơ sở dữ liệu cho các dịch vụ khác nhau.

1.3 Cơ sở lý thuyết

1.3.1 Kiến trúc Microservice

Kiến trúc microservice là phong cách kiến trúc phần mềm trong đó ứng dụng được xây dựng từ tập hợp các dịch vụ nhỏ, chạy trong tiến trình riêng lẻ, giao tiếp qua cơ chế nhẹ (HTTP API hoặc message queue). Mỗi dịch vụ được tổ chức xung quanh một khả năng nghiệp vụ cụ thể và có thể triển khai độc lập. Các đặc điểm cốt lõi gồm:

- Domain-Driven Design (DDD): mỗi service là một Bounded Context riêng biệt.
- Database-per-Service: mỗi service sở hữu CSDL riêng, không service nào truy cập trực tiếp vào DB của service khác.
- Fault Isolation: lỗi một service không kéo sập toàn bộ hệ thống.
- Independent Deployment: từng service có thể build, test, triển khai và mở rộng độc lập.

1.3.2 Cơ chế giao tiếp đồng bộ qua RESTful API

REST (Representational State Transfer) là phong cách kiến trúc cho hệ thống phân tán sử dụng giao thức HTTP. RESTful API tuân thủ: sử dụng đúng phương thức HTTP (GET, POST, PUT, PATCH, DELETE), định danh tài nguyên qua URI rõ ràng, giao tiếp stateless và trả về dữ liệu dưới dạng JSON. Trong hệ thống BookLand, RESTful API là kênh giao tiếp đồng bộ chính giữa client và các service, cũng như giữa các service với nhau khi cần dữ liệu tức thì.

1.3.3 Cơ chế giao tiếp bất đồng bộ qua Message Queue - Apache Kafka

Apache Kafka là nền tảng phân tán event streaming, thiết kế cho thông lượng cao và độ trễ thấp. Kafka hoạt động theo mô hình publish-subscribe: producer ghi message vào topic, consumer đọc message theo nhóm. Trong hệ thống phân tán,

Kafka giải quyết bài toán giao tiếp bất đồng bộ thay vì Order Service gọi trực tiếp sang Notification Service (tight coupling), Order Service chỉ publish event vào Kafka; các service liên quan tự nhận và xử lý độc lập, đảm bảo tách rời và chịu lỗi cao.

1.4 Bố cục báo cáo

Báo cáo được tổ chức thành 4 chương chính, được sắp xếp logic từ tổng quan lý thuyết đến thực hiện cụ thể và đánh giá kết quả. Cấu trúc chi tiết như sau:

CHƯƠNG 1 - Tổng quan đề tài và cơ sở lý thuyết: Trình bày bối cảnh, lý do chọn đề tài, mô tả bài toán, phạm vi hệ thống cùng các khái niệm lý thuyết nền tảng về kiến trúc Microservices, RESTful API và Apache Kafka. Chương này làm rõ mục tiêu, ý nghĩa và các thách thức kỹ thuật mà đề tài hướng đến giải quyết.

CHƯƠNG 2 - Phân tích và thiết kế hệ thống: Trình bày chi tiết quá trình phân tích yêu cầu và thiết kế kiến trúc cho hệ thống BookLand: thiết kế tổng quan hệ thống với microservices, thiết kế Database, RESTful API Endpoints. Đồng thời, chương cũng làm rõ thiết kế giao tiếp bất đồng bộ với các luồng sự kiện quan trọng và đồng bộ chỉ mục tìm kiếm, cùng với các tính năng nâng cao được áp dụng trong hệ thống.

CHƯƠNG 3 - Triển khai hệ thống: Trình bày quá trình triển khai thực tế của hệ thống BookLand: cấu hình API Gateway, triển khai các microservices bằng Docker Compose và Kubernetes, triển khai Apache Kafka cho message queue, Elasticsearch cho tìm kiếm, Redis cho caching, Supabase Storage/MinIO cho lưu trữ file, EFK Stack cho centralized logging, cùng với phần giao diện frontend. Chương cũng trình bày demo một số luồng nghiệp vụ chính để minh họa sự phối hợp giữa các service qua RESTful API và Kafka.

CHƯƠNG 4 - Thử nghiệm và đánh giá: Trình bày kết quả thử nghiệm và đánh giá tổng thể hệ thống BookLand: mô tả môi trường thử nghiệm, kết quả kiểm thử các luồng nghiệp vụ chính, đánh giá về tính đúng đắn, hiệu năng, khả năng chịu lỗi và khả năng mở rộng của hệ thống. Cuối chương là phân tích những hạn chế hiện tại của đồ án và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu

2.1.1 Yêu cầu nghiệp vụ

Hệ thống BookLand phải hỗ trợ đầy đủ luồng thương mại điện tử cho sản phẩm sách, bao gồm:

- Xác thực và phân quyền người dùng: Người dùng đăng ký, đăng nhập, quản lý hồ sơ cá nhân. Hệ thống phân biệt vai trò cơ bản như người dùng thường và quản trị. Mọi yêu cầu tới dịch vụ nội bộ phải được xác thực qua identity-service.
- Quản lý người dùng: Cung cấp API quản lý thông tin tài khoản, địa chỉ và lịch sử hoạt động. Hỗ trợ tra cứu và đồng bộ dữ liệu người dùng cho các dịch vụ khác.
- Quản lý sản phẩm sách: Quản lý danh mục sách, tác giả, nhà xuất bản, kho hàng và thông tin chi tiết sản phẩm. Hỗ trợ cập nhật, tìm kiếm và phân loại sách theo tiêu chí nghiệp vụ.
- Quản lý đơn hàng: Xử lý tạo đơn, thanh toán, cập nhật trạng thái và theo dõi lịch sử đơn hàng. Tương tác với dịch vụ người dùng, sách và sự kiện để hoàn chỉnh quy trình đặt hàng.
- Gửi thông báo: Thông báo trạng thái đơn hàng, khuyến mãi, sự kiện đến người dùng. Hỗ trợ cả giao thức bất đồng bộ và lưu trữ lịch sử thông báo.
- Quản lý sự kiện: Ghi nhận các sự kiện nghiệp vụ quan trọng như tạo đơn, thanh toán, cập nhật sách. Phục vụ mục đích phân tích, audit và kích hoạt quy trình xử lý bất đồng bộ.
- Quản lý tệp: Lưu trữ tài nguyên liên quan như ảnh bìa sách, tài liệu phê duyệt, file đính kèm. Phiên bản hiện tại dùng Supabase Storage trong File Service, đồng thời chuẩn bị cấu hình MinIO/S3-compatible cho môi trường self-hosted.

- Tìm kiếm: Hỗ trợ tìm kiếm sách và nội dung liên quan một cách hiệu quả. Tích hợp với Elasticsearch để phục vụ truy vấn toàn văn và chỉ mục.
- Chat thời gian thực: Hỗ trợ người dùng trò chuyện trực tiếp với đội ngũ hỗ trợ (admin/support) để giải quyết thắc mắc về sản phẩm, đơn hàng, thanh toán. Bao gồm quản lý cuộc trò chuyện, gửi/nhận tin nhắn real-time qua WebSocket, lưu trữ lịch sử và thông báo tin nhắn mới.

2.1.2 Yêu cầu kỹ thuật

Để đáp ứng kiến trúc microservice và môi trường vận hành, hệ thống cần thỏa mãn các yêu cầu kỹ thuật sau:

- Kiến trúc microservice: Hệ thống được chia thành 10 dịch vụ độc lập: API Gateway, Identity, User, Book, Order, Notification, Event, File, Search và Chat. Mỗi dịch vụ chịu trách nhiệm một miền nghiệp vụ riêng và có thể triển khai, mở rộng độc lập.
- Hạ tầng container và orchestration: Hỗ trợ chạy qua Docker Compose cho môi trường phát triển. Hỗ trợ triển khai trên Kubernetes (Minikube) cho môi trường thử nghiệm/production.
- Cơ sở dữ liệu và lưu trữ: MySQL dùng cho các dịch vụ quan hệ như Identity, User, Book, Order, Event và Chat. MongoDB dùng cho Notification service để lưu trữ dữ liệu phi cấu trúc. File Service hiện tích hợp Supabase Storage và có cấu hình MinIO/S3-compatible cho môi trường self-hosted. Elasticsearch dùng cho Search service để xử lý truy vấn tìm kiếm. Redis dùng cho caching, blacklist token và rate limiting khi cần.
- Messaging và bất đồng bộ: Kafka được sử dụng làm nền tảng truyền thông tin bất đồng bộ giữa các dịch vụ, đảm bảo ràng buộc dữ liệu và trao đổi sự kiện giữa các thành phần.
- API Gateway: Đóng vai trò làm điểm vào duy nhất cho frontend và các client bên ngoài, thực hiện định tuyến, xác thực, chuyển tiếp, và tổng hợp tài liệu API, giúp tách biệt frontend khỏi các dịch vụ backend nội bộ.

2.1.3 Tiêu chí chất lượng

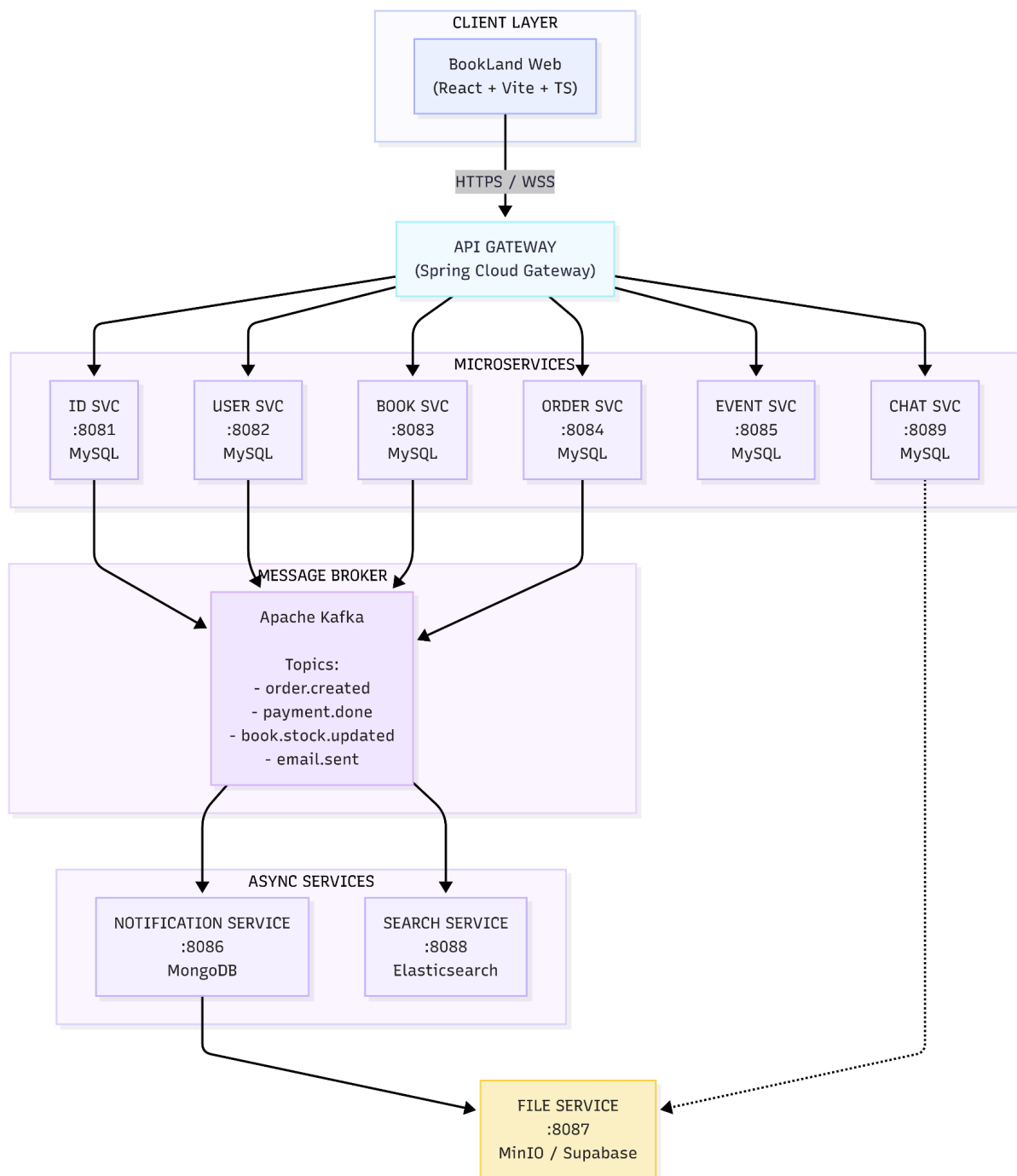
Các yêu cầu chất lượng cần được đảm bảo:

- Tính nhất quán cuối cùng
- Tính mô-đun và khả năng mở rộng: từng dịch vụ có thể mở rộng độc lập.
- Khả năng chịu lỗi: hệ thống phải chịu lỗi từng phần và tiếp tục phục vụ.
- Hiệu năng: dịch vụ tìm kiếm và đặt hàng cần đáp ứng nhanh trong điều kiện tải.
- Bảo mật: mọi endpoint quan trọng phải kiểm soát truy cập và bảo vệ dữ liệu người dùng.
- Dễ triển khai: hỗ trợ kịch bản triển khai bằng Docker Compose và khai báo manifest Kubernetes.

2.2 Thiết kế kiến trúc hệ thống

2.2.1 Kiến trúc tổng quan hệ thống

Hệ thống được tổ chức theo mô hình phân lớp: Client Layer - API Gateway - Microservice Layer - Infrastructure Layer. Tất cả request từ client đều đi qua API Gateway, nơi thực hiện xác thực JWT, rate limiting và định tuyến.



Hình 2.1 Sơ đồ kiến trúc tổng quan

Hai luồng giao tiếp chính trong hệ thống:

- Giao tiếp đồng bộ (REST): Client ↔ Gateway ↔ Service; Order Service ↔ Book Service (kiểm tra và cập nhật tồn kho).

- Giao tiếp bất đồng bộ (Kafka): Order Service → Notification Service (gửi thông báo/email); Book Service → Search Service (đồng bộ Elasticsearch index); Chat Service → Notification Service để xử lý thông báo tin nhắn mới.

2.2.2 Thiết kế các microservice

Hệ thống áp dụng nguyên tắc Domain-Driven Design để phân rã thành 9 dịch vụ, mỗi dịch vụ tương ứng với một Bounded Context nghiệp vụ riêng biệt

2.2.2.1 API Gateway

Đây là cổng vào duy nhất của toàn bộ hệ thống, được xây dựng trên nền tảng Spring Cloud Gateway với kiến trúc reactive non-blocking. Tất cả request từ client đều phải đi qua đây trước khi được định tuyến tới service tương ứng

API Gateway chịu trách nhiệm thực thi các chính sách an toàn, kiểm soát lưu lượng và bảo vệ hạ tầng nội bộ thông qua nhiều cơ chế tích hợp nâng cao. Hệ thống cấu hình bộ lọc Rate Limiting sử dụng Redis làm backend lưu trữ trạng thái; trong mã nguồn hiện tại, route Identity Service được giới hạn ở mức 10 request/giây với burst capacity 20. Phân hệ này cũng đã thêm dependency và cấu hình nền cho Resilience4j, tuy nhiên chưa gắn CircuitBreaker filter đầy đủ cho toàn bộ route, nên cơ chế ngắt mạch vẫn là hướng hoàn thiện tiếp theo. Bên cạnh đó, Gateway còn đảm nhiệm việc cấu hình CORS, tổng hợp Swagger/OpenAPI của các service và định tuyến toàn bộ request từ frontend vào hệ thống backend.

Cơ chế bảo mật tại API Gateway được phân tách rõ ràng dựa trên các bộ quy tắc lọc đường dẫn cụ thể thông qua thành phần AuthenticationFilter.

- Đối với các tài nguyên và endpoint mang tính chất công khai như quy trình đăng nhập, đăng ký tài khoản, làm mới mã token, các cổng phản hồi giao dịch thanh toán, cũng như các tác vụ truy vấn hiển thị sản phẩm, hệ thống cho phép yêu cầu đi qua trực tiếp không cần xác thực.
- Ngược lại, đối với các endpoint bảo mật yêu cầu định danh, bộ lọc sẽ trích xuất mã token định dạng Bearer từ Header, kích hoạt một lệnh gọi đồng bộ sang

Identity Service để kiểm tra tính hợp lệ. Ngay khi token được xác nhận thành công, Gateway sẽ tự động inject các siêu dữ liệu thông tin của người dùng vào HTTP Header downstream dưới dạng các biến X-User-Id, X-User-Email và X-User-Roles, giúp các microservice nội bộ dễ dàng nhận biết và phân quyền xử lý nghiệp vụ mà không cần thực hiện xác thực lại từ đầu.

2.2.2.2 Identity Service

Identity Service là dịch vụ chịu trách nhiệm xác thực và phân quyền, sử dụng JWT (access token hết hạn sau 1 giờ, refresh token 7 ngày) và hỗ trợ đăng nhập qua Google OAuth2.

Khi người dùng đăng ký tài khoản bằng email và mật khẩu, dịch vụ sẽ thực hiện mã hóa bảo mật thông tin bằng thuật toán BCrypt trước khi lưu trữ. Bên cạnh quy trình đăng nhập và phát hành cặp mã token, phân hệ này còn xử lý nghiệp vụ đăng xuất bằng cách đưa các token hiện hành vào danh sách đen lưu trữ tại Redis với thời gian sống (TTL) tương ứng với thời điểm hết hạn của token đó. Ngoài ra, dịch vụ cung cấp API Introspect để kiểm tra tính hợp lệ của chữ ký JWT, rà soát blacklist và trả về các siêu dữ liệu định danh như userId, roles, email cho API Gateway, đồng thời quản lý toàn diện các thao tác khởi tạo, cập nhật quyền hạn (CRUD Role và Permission) trong hệ thống.

2.2.2.3 User Service

User Service quản lý thông tin cá nhân người dùng, địa chỉ giao hàng và danh sách sách yêu thích. Service này tách biệt khỏi Identity Service theo nguyên tắc Single Responsibility: Identity Service quản lý thông tin xác thực, User Service quản lý thông tin nghiệp vụ người dùng.

- Quản lý profile: tên, avatar, số điện thoại, ngày sinh.
- Quản lý địa chỉ giao hàng (CRUD, hỗ trợ nhiều địa chỉ).
- Danh sách yêu thích (Wishlist): thêm, xóa, xem danh sách.

2.2.2.4 Book Service

Book Service là service trung tâm của nghiệp vụ thương mại điện tử, chịu trách nhiệm quản lý toàn bộ catalog sách cùng các thực thể liên quan.

Chức năng chính của service này bao gồm các thao tác CRUD sách, tác giả, danh mục, nhà xuất bản, series và nhà cung cấp. Bên cạnh việc quản lý chi tiết hệ thống tồn kho và quy trình nhập kho thông qua hóa đơn, dịch vụ còn xử lý các tính năng tương tác người dùng như review và bình luận sách, đồng thời cung cấp các API internal để hỗ trợ Order Service kiểm tra, cập nhật số lượng tồn kho theo thời gian thực.

Đặc biệt, để đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, Book Service sẽ xuất bản các sự kiện lên Kafka Broker khi có biến động dữ liệu để các dịch vụ khác như Search Service và Notification Service cùng tiêu thụ. Các sự kiện này được cấu hình tường minh qua ba Topic chính: book.created mang gói tin định danh và thông tin sách để lập chỉ mục tìm kiếm; book.stock.updated truyền tải trạng thái kho thay đổi; và cuối cùng là book.deleted chỉ chứa bookId nhằm mục đích đồng bộ xóa bỏ tài nguyên trên toàn hệ thống.

2.2.2.5 Order Service

Order Service là service phức tạp nhất trong hệ thống, xử lý toàn bộ luồng từ quản lý giỏ hàng đến tạo đơn hàng, tích hợp thanh toán VNPay và publish Kafka events kích hoạt các service downstream.

Dịch vụ cung cấp các chức năng cốt lõi bao gồm thêm, xóa, cập nhật số lượng sản phẩm trong giỏ hàng, đồng thời quản lý các danh mục phụ trợ như phương thức thanh toán và phương thức vận chuyển. Khi tiến hành đặt hàng, hệ thống sẽ khởi tạo hóa đơn và theo dõi dịch chuyển trạng thái đơn hàng theo sơ đồ tuyến tính PENDING → CONFIRMED → SHIPPING → DELIVERED/CANCELLED. Nhằm tối ưu hóa trải nghiệm giao dịch trực tuyến, dịch vụ được tích hợp với cổng thanh toán VNPay để

tự động khởi tạo URL thanh toán an toàn và tiếp nhận, xử lý các phản hồi bất đồng bộ thông qua callback IPN.

Đặc biệt, để đảm bảo tính nhất quán dữ liệu và kích hoạt các dịch vụ downstream, ngay sau khi đơn hàng được tạo mới hoặc thay đổi trạng thái thành công, tầng nghiệp vụ BillService sẽ ngay lập tức kích hoạt bộ phát sự kiện nội bộ mang tên NotificationProducer. Thành phần này có nhiệm vụ đóng gói thông tin và xuất bản một sự kiện dữ liệu cấu trúc vào Kafka Topic notification-events. Thông điệp này sau đó sẽ được tiêu thụ bởi Notification Service để thực hiện các tác vụ gửi email xác nhận và đẩy thông báo thời gian thực đến máy khách, hoàn thiện một chu trình xử lý hướng sự kiện khép kín.

2.2.2.6 Notification Service

Notification Service được thiết kế theo mô hình dịch vụ tiêu thụ thuần túy (pure consumer) - không khởi tạo bất kỳ lệnh gọi REST API đồng bộ nào sang các dịch vụ khác mà chỉ lắng nghe và xử lý các sự kiện từ Kafka Broker. Đây là minh chứng rõ ràng nhất cho tính tách rời trong kiến trúc event-driven.

Ở tầng xử lý dữ liệu trung gian, dịch vụ này cấu hình các bộ tiêu thụ thông điệp thuộc nhóm chung notification-group để lắng nghe đồng thời trên 2 Topic cốt lõi.:

- Topic notification-events: tiếp nhận các thông điệp nghiệp vụ để tự động khởi tạo dữ liệu thông báo in-app lưu trữ vào cơ sở dữ liệu MongoDB, kích hoạt luồng đẩy dữ liệu thời gian thực qua giao thức WebSocket và tự động biên dịch, gửi email định dạng HTML template nếu cờ dữ liệu sendEmail được thiết lập trạng thái true.
- Topic email-events, chuyên phục vụ riêng cho các tác vụ gửi email thông thường dưới dạng văn bản hoặc HTML tùy biến.

Notification Service còn tích hợp giải pháp Spring WebSocket dựa trên giao thức STOMP; cho phép các ứng dụng Client chủ động subscribe vào các kênh định danh chuyên biệt, từ đó trực tiếp tiếp nhận các gói tin thông báo tức thì từ hệ thống

backend mà không cần sử dụng đến các giải pháp truy vấn lặp lại liên tục gây lãng phí tài nguyên mạng.

2.2.2.7 Event Service

Event Service quản lý các chương trình sự kiện và khuyến mãi: tạo sự kiện, định nghĩa rule áp dụng (theo danh mục, tác giả, hoặc toàn bộ), xác định đối tượng áp dụng, và hành động (EventAction - giảm giá %, giảm tiền cố định). Service này cung cấp RESTful API cho admin quản lý và cho Order Service truy vấn khi tính giá đơn hàng.

2.2.2.8 File Service

File Service đảm nhiệm việc upload, lưu trữ và phục vụ hình ảnh/tài liệu cho toàn bộ hệ thống. Trong mã nguồn hiện tại, controller upload sử dụng SupabaseStorageService để đưa ảnh lên Supabase Storage, đồng thời service vẫn có cấu hình MinIO/S3-compatible để phục vụ kịch bản self-hosted. Service cung cấp API upload (multipart/form-data), lấy danh sách ảnh, trả về URL truy cập file và xóa file.

2.2.2.9 Search Service

Search Service cung cấp khả năng tìm kiếm sách full-text nâng cao, sử dụng Elasticsearch 8.x làm kho lưu trữ dữ liệu duy nhất. Dịch vụ này hoạt động theo mô hình tiêu thụ thông điệp thuần túy từ Kafka Broker để tự động đồng bộ hóa trạng thái dữ liệu.

Cụ thể, thông qua hệ thống Kafka Consumers, Search Service lắng nghe đồng thời trên ba Topic cốt lõi:

- Tiếp nhận sự kiện từ topic book.created để tiến hành lập chỉ mục tài liệu sách mới vào Elasticsearch;
- Cập nhật nhanh số lượng sản phẩm trong kho;
- Thực hiện xóa bỏ hoàn toàn tài liệu khỏi chỉ mục tìm kiếm ngay khi nhận được sự kiện hủy tài nguyên.

Ngoài ra, dịch vụ cung cấp Search API mạnh mẽ, cho phép thực hiện truy vấn full-text theo tên sách, tác giả hoặc danh mục. Các API này được tích hợp các tính năng lọc nâng cao theo khoảng giá, kiểm tra trạng thái tồn kho, hỗ trợ phân trang tối ưu kết hợp cùng cơ chế sắp xếp đa tiêu chí như theo giá cả, mức độ bán chạy hoặc sản phẩm mới nhất, mang lại trải nghiệm tìm kiếm mượt mà và chính xác cho khách hàng.

2.2.2.10 Chat Service

Service này đóng vai trò quan trọng trong việc nâng cao trải nghiệm người dùng, giúp giải quyết nhanh chóng các thắc mắc liên quan đến đơn hàng, sản phẩm, thanh toán và các vấn đề khác.

Các chức năng chính bao gồm tạo cuộc chat mới giữa user và admin, hỗ trợ nhiều phiên chat đồng thời, xử lý tin nhắn thời gian thực bằng cách WebSocket kết hợp giao thức STOMP để truyền nhận tin nhắn hai chiều với độ trễ thấp, tin nhắn được lưu trữ bền vững trong cơ sở dữ liệu riêng. Tin nhắn được phân quyền - người dùng chỉ có thể chat với admin/support; admin có thể quản lý nhiều cuộc trò chuyện cùng lúc, đánh dấu trạng thái tin nhắn (đã đọc/chưa đọc) và khi có tin nhắn mới, service publish event lên Kafka để Notification Service đẩy thông báo realtime đến các bên liên quan.

Ngoài ra, tất cả kết nối WebSocket đều phải thông qua JWT Authentication Filter tại API Gateway. Mỗi tin nhắn đều được kiểm tra quyền sở hữu cuộc trò chuyện.

2.2.2.10 Bảng phân rã dịch vụ theo domain-driven design

Bảng 2.1 Bảng phân rã dịch vụ theo domain-driven design

#	Service	Port	Database	Nghịệp vụ chính
1	API Gateway	8080	-	Cổng vào, JWT Filter, Rate Limit, Routing

2	Identity Service	8081	MySQL (identity_db)	Đăng ký, đăng nhập, JWT, OAuth2 Google
3	User Service	8082	MySQL (user_db)	Profile, địa chỉ, danh sách yêu thích
4	Book Service	8083	MySQL (book_db)	Sách, tác giả, danh mục, tồn kho, review
5	Order Service	8084	MySQL (order_db)	Giỏ hàng, đơn hàng, VNPAY, vận chuyển
6	Notification Service	8086	MongoDB (notification_db)	Thông báo, email, WebSocket realtime
7	Event Service	8085	MySQL (event_db)	Sự kiện, khuyến mãi, banner
8	File Service	8087	Supabase Storage / MinIO S3-compatible	Upload, lưu trữ file và hình ảnh
9	Search Service	8088	Elasticsearch	Tìm kiếm sách full-text nâng cao
10	Chat Service	8089	MySQL (chat_db)	Chat thời gian thực, Lịch sử tin nhắn

2.3 Thiết kế Database - Database per Service

Nguyên tắc cốt lõi: mỗi service có cơ sở dữ liệu hoàn toàn riêng biệt. Không service nào được phép truy cập trực tiếp vào database của service khác. Mọi nhu cầu dữ liệu liên service đều thông qua REST API hoặc Kafka event. Trong đó Notification Service dùng MongoDB vì dữ liệu thông báo có cấu trúc linh hoạt (schema-less) và cần truy vấn theo userId nhanh; Search Service dùng Elasticsearch vì tính năng

full-text inverted index; File Service dùng Supabase Storage trong bản hiện tại và chuẩn bị MinIO/S3-compatible cho object storage self-hosted.

Bảng 2.2 thiết kế database-per-service

Service	Loại DB	Tên DB	Các bảng/collection chính
Identity Service	MySQL 8.0	identity_db	users, roles, permissions, invalidated_tokens
User Service	MySQL 8.0	user_db	user_profiles, addresses, wishlists
Book Service	MySQL 8.0	book_db	books, authors, categories, publishers, series, book_comments, purchase_invoices
Order Service	MySQL 8.0	order_db	bills, bill_books, carts, cart_items, payment_methods, shipping_methods, payment_transactions
Event Service	MySQL 8.0	event_db	events, event_rules, event_targets, event_actions, event_logs
Notification Service	MongoDB 7.0	notification_db	notifications, chat_messages
Search Service	Elasticsearch 8.11	(index)	books (Elasticsearch document)
File Service	Supabase Storage / MinIO	(buckets)	bookland-images, bookland-files

Chat Service	MySQL 8.0	chat_db	conversations, messages, participants
--------------	-----------	---------	---------------------------------------

2.4 Thiết kế RESTful API

2.4.1 Nguyên tắc thiết kế

Hệ thống BookLand thiết lập một chuẩn chung nghiêm ngặt cho việc xây dựng và phát triển giao tiếp đồng bộ. Tất cả các giao diện lập trình ứng dụng (API) giữa các dịch vụ và client đều phải tuân thủ hệ thống nguyên tắc cốt lõi sau:

- Chuẩn hóa định danh tài nguyên và hành động: Sử dụng danh từ (nouns) để định danh tài nguyên trong cấu trúc URI, sử dụng các phương thức HTTP tiêu chuẩn (verbs) để thể hiện rõ ràng hành động tác động. Trong đó, GET để truy vấn dữ liệu (đảm bảo tính an toàn và bất biến - idempotent), POST để khởi tạo tài nguyên mới, PUT/PATCH để cập nhật thông tin và DELETE để xóa bỏ tài nguyên.
- Tối ưu hóa hiệu năng truy vấn phân trang: Sử dụng các tham số truy vấn bao gồm page và size để thực hiện phân trang và giới hạn dữ liệu ở tầng cơ sở dữ liệu, tránh hiện tượng quá tải đường truyền mạng.
- Đồng nhất cấu trúc phản hồi (Response Wrapper): Nhằm đơn giản hóa quá trình xử lý dữ liệu ở phía Client, toàn bộ các API trong hệ thống đều trả về một cấu trúc dữ liệu bao bọc thống nhất dưới định dạng JSON.
- Bảo mật tầng giao tiếp: Cơ chế xác thực và phân quyền được thực thi nhất quán qua việc kiểm tra mã JSON Web Token (JWT) định dạng Bearer, được đính kèm trực tiếp trong thuộc tính `Authorization` của HTTP Request Header đối với mọi yêu cầu truy cập tài nguyên bảo mật.
- Tự động hóa tài liệu kỹ thuật: Mỗi microservice được tích hợp thư viện SpringDoc nhằm tự động hóa việc biên dịch và tài liệu hóa các endpoint theo đặc tả Swagger/OpenAPI 3.0 trực quan.

2.4.2 Bảng tổng hợp API Endpoints

2.4.2.1 Identity Service - /auth/**

Bảng 2.3 Identity Service API Endpoints

Method	Endpoint	Mô tả	Auth
POST	/auth/register	Đăng ký tài khoản mới	Public
POST	/auth/login	Đăng nhập, nhận JWT token	Public
POST	/auth/admin/login	Đăng nhập dành cho Admin	Public
POST	/auth/google	Đăng nhập Google OAuth2	Public
POST	/auth/refresh	Làm mới access token	Public
POST	/auth/logout	Đăng xuất (blacklist token)	Bearer
POST	/auth/introspect	Kiểm tra token hợp lệ (internal)	Bearer
GET	/api/roles	Danh sách role	Admin
POST	/api/roles	Tạo role mới	Admin
DELETE	/api/roles/{name}	Xóa role	Admin

2.4.2.2 User Service — /api/users, /api/addresses, /api/wishlists

Bảng 2.4 User Service API Endpoints

Method	Endpoint	Mô tả	Auth
GET	/api/users/me	Lấy thông tin profile cá nhân	Bearer
PUT	/api/users/me	Cập nhật profile	Bearer
GET	/api/users	Danh sách tất cả user	Admin

GET	/api/users/{id}	Chi tiết user (Admin)	Admin
POST	/api/addresses	Thêm địa chỉ giao hàng	Bearer
GET	/api/addresses	Danh sách địa chỉ của tôi	Bearer
PUT	/api/addresses/{id}	Cập nhật địa chỉ	Bearer
DELETE	/api/addresses/{id}	Xóa địa chỉ	Bearer
POST	/api/wishlists/{bookId}	Thêm sách vào yêu thích	Bearer
GET	/api/wishlists	Danh sách sách yêu thích	Bearer
DELETE	/api/wishlists/{bookId}	Xóa khỏi yêu thích	Bearer

2.4.2.3 Book Service — /api/books, /api/authors, /api/categories...

Bảng 2.5 Book Service API Endpoints

Method	Endpoint	Mô tả	Auth
GET	/api/books	Danh sách sách (filter, paginate)	Public
GET	/api/books/{id}	Chi tiết sách	Public
GET	/api/books/best-sellers	Sách bán chạy nhất	Public
POST	/api/books	Thêm sách mới	Admin
PUT	/api/books/{id}	Cập nhật sách	Admin
DELETE	/api/books/{id}	Xóa sách	Admin
POST	/api/books/{id}/comments	Đánh giá sách	Bearer
GET	/api/books/{id}/comments	Đọc đánh giá	Public
DELETE	/api/books/{id}/comments/{commentId}	Xóa đánh giá	Owner/Admin

POST	/api/purchase-invoices	Nhập kho	Admin
GET	/api/categories	Danh sách danh mục	Public
GET	/api/authors	Danh sách tác giả	Public
GET	/api/publishers	Danh sách nhà xuất bản	Public
GET	/api/internal/books/{id}/stock	Kiểm tra tồn kho (internal)	Service
PUT	/api/internal/books/{id}/stock	Cập nhật tồn kho (internal)	Service

2.4.2.4 Order Service — /api/carts, /api/bills, /vnpay/**

Bảng 2.6 Order Service API Endpoints

Method	Endpoint	Mô tả	Auth
GET	/api/carts/my	Lấy giỏ hàng của tôi	Bearer
POST	/api/carts/my/items	Thêm sách vào giỏ hàng	Bearer
PUT	/api/carts/my/items/{id}	Cập nhật số lượng trong giỏ	Bearer
DELETE	/api/carts/my/items/{id}	Xóa item khỏi giỏ hàng	Bearer
DELETE	/api/carts/my	Xóa toàn bộ giỏ hàng	Bearer
POST	/api/bills	Tạo đơn hàng mới từ giỏ	Bearer
GET	/api/bills	Danh sách đơn hàng (Admin, filter)	Admin
GET	/api/bills/my	Đơn hàng của tôi	Bearer
GET	/api/bills/{id}	Chi tiết đơn hàng	Bearer

PATCH	/api/bills/{id}/status	Cập nhật trạng thái đơn hàng	Admin
POST	/vnpay/create-payment	Tạo URL thanh toán VNPay	Bearer
GET	/vnpay/payment_infor	Callback xác nhận thanh toán	Public
GET	/api/payment-methods	Danh sách phương thức thanh toán	Public
GET	/api/shipping-methods	Danh sách phương thức vận chuyển	Public

2.4.2.5 Notification Service — /api/notifications, /ws/**

Bảng 2.7 Notification Service API Endpoints

Method	Endpoint	Mô tả	Auth
GET	/api/notifications	Danh sách thông báo của tôi	Bearer
PATCH	/api/notifications/{id}/read	Đánh dấu đã đọc	Bearer
PATCH	/api/notifications/read-all	Đánh dấu tất cả đã đọc	Bearer
POST	/api/notifications/direct	Gửi thông báo trực tiếp (Admin)	Admin
WS	/ws (STOMP)	WebSocket connection endpoint	Bearer
SUB	/queue/notifications/{userId}	Subscribe nhận notification realtime	Bearer

2.4.2.6 Search Service — /api/search/**

Bảng 2.8 Search Service API Endpoints

Method	Endpoint	Mô tả	Auth
GET	/api/search/books	Tìm kiếm sách full-text (Elasticsearch)	Public
GET	/api/search/books/suggest	Gợi ý từ khóa tìm kiếm	Public

2.4.2.7 Chat Service — /api/chat/**

Bảng 2.9 Chat Service API Endpoints

Method	Endpoint	Mô tả	Auth
POST	/api/chat/conversations	Tạo cuộc trò chuyện mới	Bearer
GET	/api/chat/conversations	Lấy danh sách cuộc trò chuyện của tôi	Bearer
GET	/api/chat/conversations/{id}	Chi tiết cuộc trò chuyện	Bearer
GET	/api/chat/conversations/{id}/messages	Lấy lịch sử tin nhắn theo phân trang	Bearer
POST	/api/chat/conversations/{id}/messages	Gửi tin nhắn mới	Bearer
PATCH	/api/chat/messages/{messageId}/read	Đánh dấu tin nhắn đã đọc	Bearer
GET	/api/chat/admin/conversations	Admin: Lấy tất cả cuộc trò chuyện	Admin
PATCH	/api/chat/conversations/{id}/status	Admin: Cập nhật trạng thái cuộc chat	Admin

WS	/ws	WebSocket endpoint (STOMP)	Bearer
SUB	/queue/chat/{conversationId}	Subscribe nhận tin nhắn realtime	Bearer
SUB	/topic/notifications	Subscribe thông báo tin nhắn mới	Bearer

2.5 Giao tiếp bất đồng bộ qua Message Queue (Kafka)

2.5.1 Tổng quan Kafka trong hệ thống

Apache Kafka được triển khai với Confluent Platform 7.5, sử dụng Zookeeper để quản lý cluster. Kafka UI được tích hợp để giám sát topic, consumer group và message flow. Hệ thống định nghĩa các Kafka topic trong bảng sau:

Bảng 2.10 Các kafka topic trong hệ thống

Topic	Producer	Consumer	Mô tả
notification-events	Order Service	Notification Service	Thông báo đặt hàng, cập nhật trạng thái
email-events	Identity Service	Notification Service	Gửi email xác thực, chào mừng
book.created	Book Service	Search Service	Index sách mới vào Elasticsearch
book.stock.updated	Book Service	Search Service	Cập nhật tồn kho trong index
book.deleted	Book Service	Search Service	Xóa sách khỏi Elasticsearch index

chat-events	Chat Service	Notification Service	Thông báo tin nhắn chat mới (realtime)
-------------	--------------	----------------------	--

2.5.3 Luồng đặt hàng & Thanh toán

Đây là luồng nghiệp vụ phức tạp nhất trong hệ thống, kết hợp cả giao tiếp đồng bộ (REST) và bất đồng bộ (Kafka):

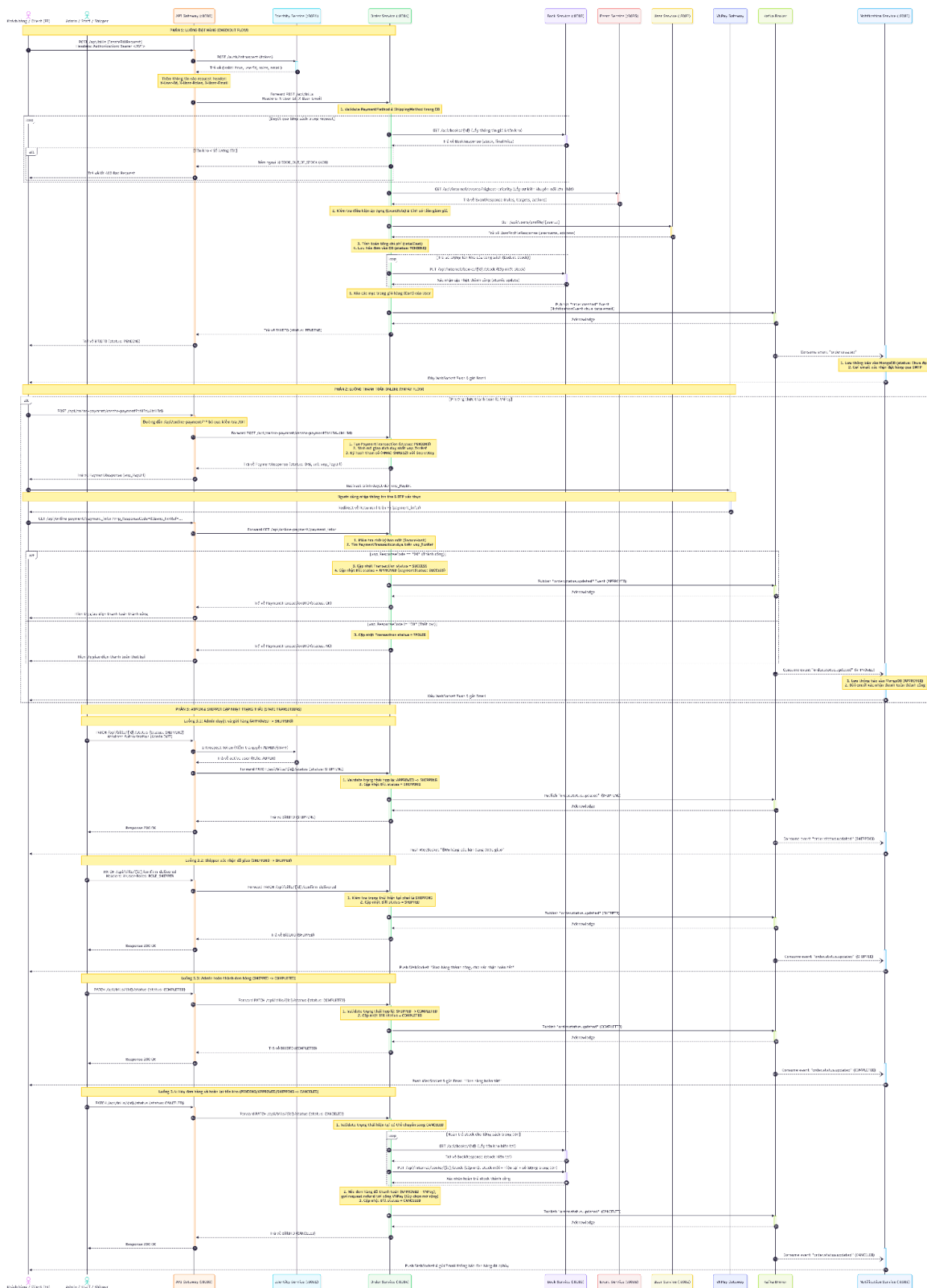
- Order Service tạo đơn hàng → Publish notification-events.
- Notification Service xử lý gửi email + push notification.

Hình ảnh dưới đây là luồng đặt hàng và thanh toán, cụ thể file .svg ở phần phụ lục dự án:

Trước hết, người dùng đăng nhập vào hệ thống và lựa chọn sách cần mua trên giao diện frontend. Khi người dùng thêm sách vào giỏ hàng, request được gửi qua API Gateway đến Order Service. Order Service lưu thông tin giỏ hàng và các mặt hàng tương ứng trong order_db.

Khi người dùng xác nhận tạo đơn hàng, Order Service kiểm tra thông tin sách và số lượng tồn kho thông qua Book Service. Nếu dữ liệu hợp lệ, Order Service tạo hóa đơn, lưu thông tin đơn hàng, cập nhật trạng thái nghiệp vụ và xử lý thông tin thanh toán hoặc phương thức vận chuyển. Trong trường hợp có chương trình khuyến mãi, Order Service có thể gọi Event Service để lấy sự kiện đang áp dụng và tính toán giá trị giảm giá phù hợp.

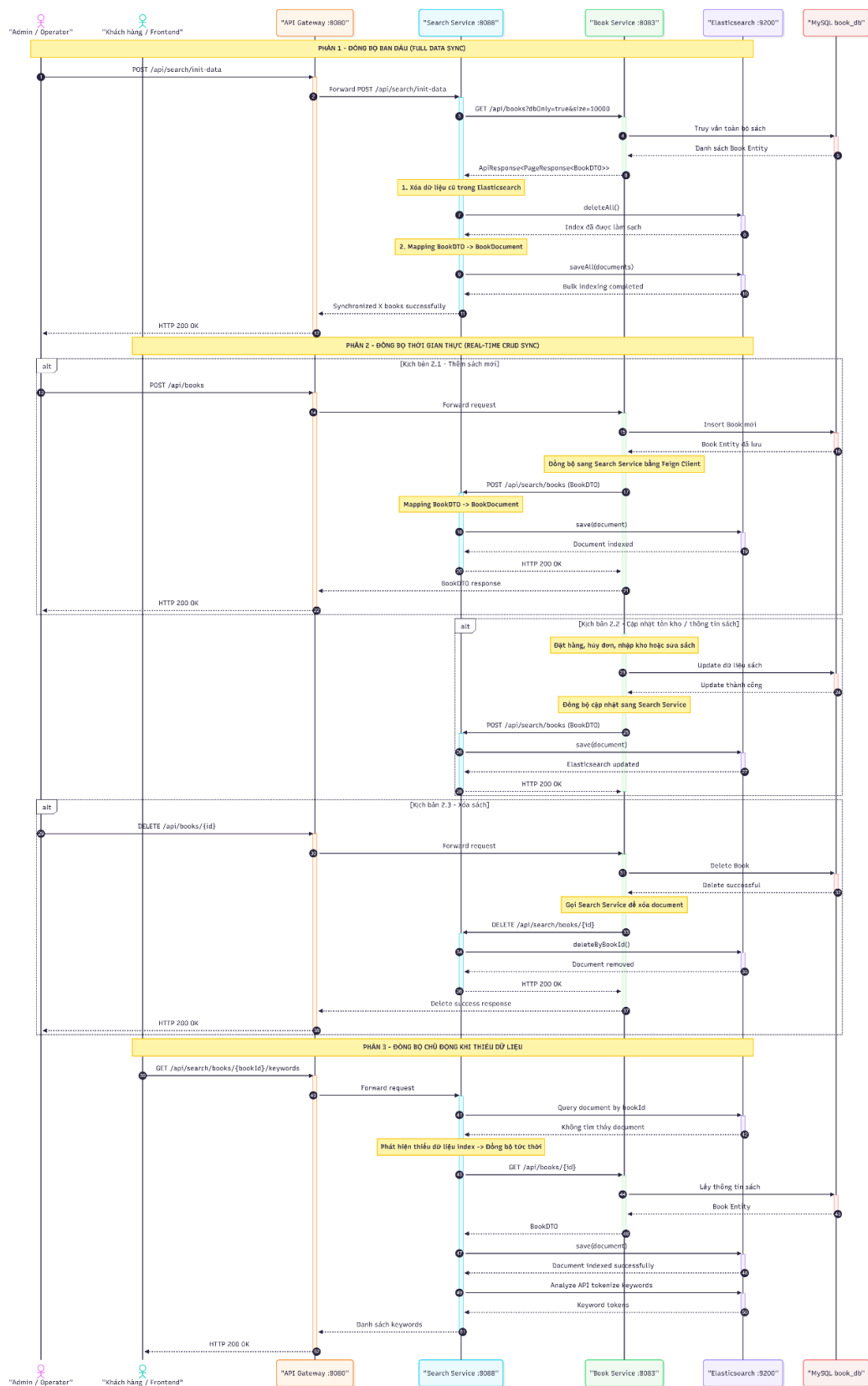
Sau khi đơn hàng được tạo hoặc cập nhật trạng thái thành công, Order Service không gửi email hoặc thông báo trực tiếp. Thay vào đó, service này đóng gói dữ liệu sự kiện và publish message vào Kafka topic notification-events. Notification Service lắng nghe topic này, nhận message, tạo bản ghi thông báo trong MongoDB, đẩy thông báo realtime qua WebSocket và gửi email nếu sự kiện yêu cầu. Cách thiết kế này giúp luồng đặt hàng không bị phụ thuộc vào thời gian xử lý thông báo, đồng thời làm giảm sự phụ thuộc trực tiếp giữa Order Service và Notification Service.



Hình 2.2 Order Flow

2.5.4 Luồng đồng bộ Search Index

Khi admin thêm sách mới hoặc cập nhật tồn kho, Book Service publish Kafka event; Search Service consume và cập nhật Elasticsearch index tự động, đảm bảo search index luôn đồng bộ với Book Service mà không cần gọi REST trực tiếp.



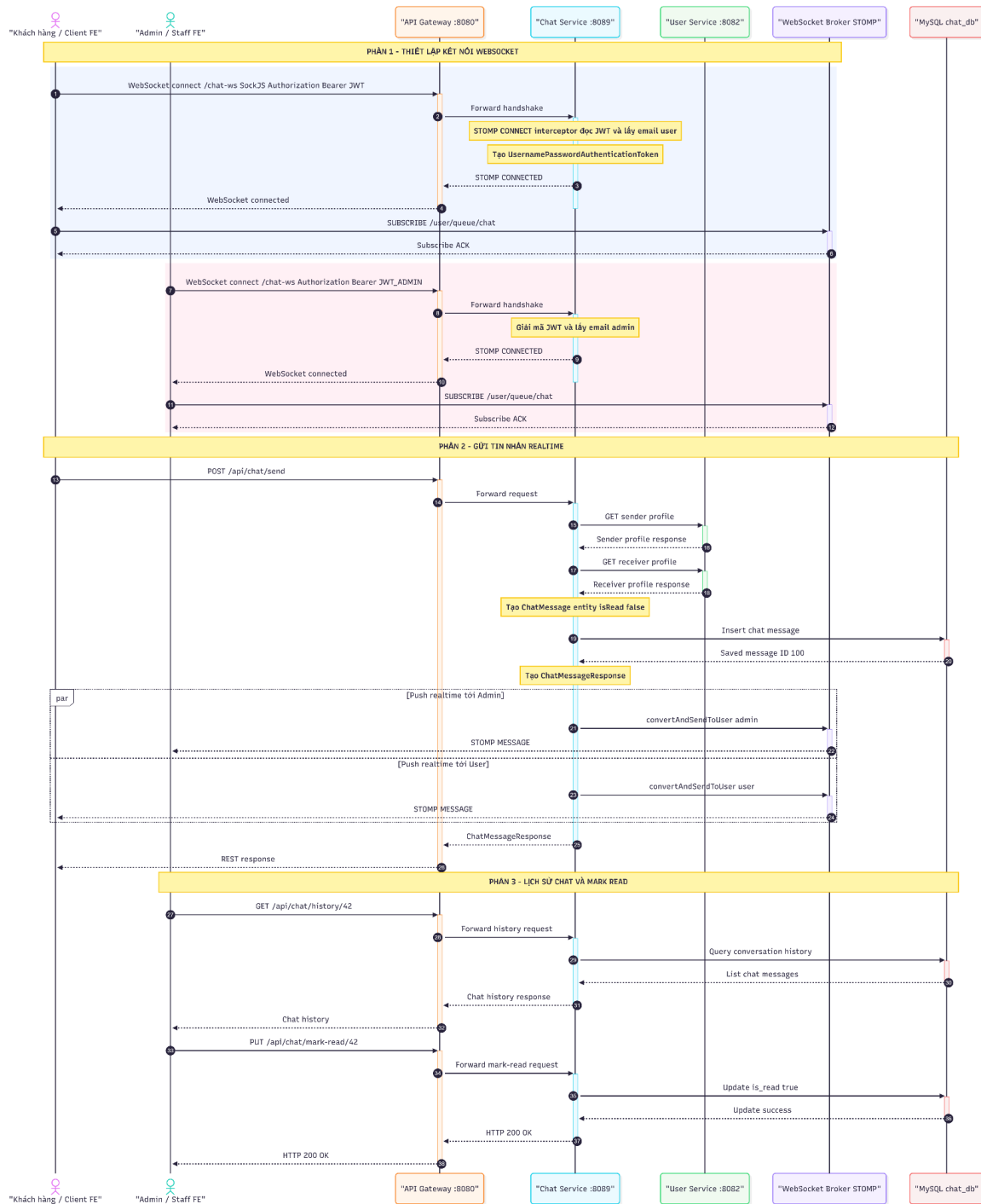
Hình 2.3 Book Flow

2.5.5 Luồng xử lý tin nhắn

Khi người dùng gửi một tin nhắn mới trong cuộc trò chuyện:

1. Chat Service lưu tin nhắn vào chat_db.
2. Publish event chat-events chứa thông tin: conversationId, senderId, receiverId, content, timestamp.
3. Notification Service consume event →
 - Đẩy thông báo realtime qua WebSocket cho người nhận.
 - Gửi email thông báo nếu người nhận không online.
4. Cập nhật trạng thái "đã nhận" / "đã đọc" cho người gửi.

Luồng này đảm bảo tính bất đồng bộ và không làm block Chat Service khi gửi thông báo.



Hinh Chat Flow

2.5.5 Xử lý lỗi trong Kafka

Cơ chế xử lý lỗi được thiết kế chặt chẽ ở cả hai đầu phát và nhận nhằm bảo đảm tính toàn vẹn dữ liệu và ngăn chặn tình trạng gián đoạn hệ thống. Tại tầng tiêu thụ, các phương thức được đánh dấu với `@KafkaListener` được bao bọc bởi các khối lệnh try-catch ngoại lệ tập trung; khi có sự cố phát sinh, thông tin chi tiết về lỗi sẽ được ghi nhận vào hệ thống log mà hoàn toàn không làm sụp đổ tiến trình chạy ngầm của consumer. Trong luồng xử lý thông báo, nếu việc gửi email gặp thất bại (user email không lấy được), hệ thống log warning và chủ động tiếp tục thực hiện nghiệp vụ khởi tạo thông báo nội bộ (notification in-app) nhằm duy trì chức năng cốt lõi.

Ngoài ra, hệ thống tận dụng tối đa cơ chế tự động thử lại (retry delivery) của Kafka Broker nếu phía consumer chưa gửi tín hiệu xác nhận đã xử lý thành công (acknowledge). Đối với tầng phát tin, các khối lệnh try-catch cũng được tích hợp trực tiếp bên trong thành phần NotificationProducer, cho phép kịp thời phát hiện và ghi vết nhật ký lỗi khi xảy ra sự cố không thể xuất bản thông điệp lên các Topic trung gian, đảm bảo khả năng kiểm soát luồng dữ liệu một cách minh bạch.

2.6 Các tính năng nâng cao

Ngoài các yêu cầu bắt buộc, hệ thống BookLand có triển khai một số thành phần hỗ trợ vận hành như API Gateway, Docker Compose, Kubernetes manifest, Kafka UI, Swagger/OpenAPI, tích hợp thanh toán VNPay, rate limiting bằng Redis và centralized logging bằng EFK Stack. Một số cơ chế nâng cao như circuit breaker/retry policy toàn diện và distributed tracing vẫn được xác định là hướng mở rộng tiếp theo, do hiện tại mới có cấu hình nền và một số fallback thủ công trong từng service.

2.6.1 API Gateway — Spring Cloud Gateway

API Gateway được triển khai đầy đủ với Spring Cloud Gateway (reactive, non-blocking), đóng vai trò Single Entry Point, giảm coupling giữa client và các microservice. Toàn bộ logic xác thực được tập trung tại Gateway, các service downstream chỉ cần tin tưởng header đã được inject.

2.6.2 Xử lý lỗi cơ bản và định hướng Circuit Breaker

Trong phiên bản hiện tại, hệ thống đã có các lớp xử lý ngoại lệ tập trung tại từng service, cơ chế ghi log khi gọi service khác thất bại và fallback thủ công trong một số luồng nghiệp vụ. Ví dụ, Book Service có thể chuyển sang truy vấn database khi Search Service hoặc Elasticsearch không phản hồi. API Gateway đã có dependency Spring Cloud CircuitBreaker Resilience4j và cấu hình instance circuit breaker cho book-service, tuy nhiên chưa gắn CircuitBreaker filter/fallback URI đầy đủ vào các route và chưa có retry policy thống nhất cho toàn bộ lời gọi liên service. Vì vậy, phần này được đánh giá là đã có nền tảng kỹ thuật và fallback cục bộ, nhưng circuit breaker/retry policy hoàn chỉnh vẫn là hướng phát triển tiếp theo.

2.6.3 Redis và định hướng Rate Limiting

Rate Limiting được triển khai tại API Gateway thông qua Spring Cloud Gateway RequestRateLimiter và Redis Token Bucket Algorithm. Trong cấu hình hiện tại, route Identity Service được giới hạn 10 request/giây với burst capacity 20 request. Đây là cơ chế bảo vệ bước đầu cho các API nhạy cảm như đăng nhập, đăng ký, quản lý người dùng và quyền hạn; các route còn lại có thể tiếp tục mở rộng cấu hình rate limit theo mức độ rủi ro của từng nghiệp vụ.

2.6.4 Centralized Logging

Tác dụng của Centralized Logging trong kiến trúc Microservices Trong kiến trúc Microservices, hệ thống được chia nhỏ thành nhiều dịch vụ hoạt động độc lập và phân tán trên các container/pod khác nhau. Việc giám sát và dò lỗi (debug) theo phương pháp truyền thống như SSH vào từng máy chủ ảo hoặc chạy lệnh đọc log thủ công (docker logs hay kubectl logs) trở nên bất khả thi và cực kỳ tốn thời gian khi quy mô hệ thống phình to. Giải pháp Centralized Logging (Quản lý log tập trung) ra đời nhằm thu thập, lưu trữ và hợp nhất toàn bộ dữ liệu log từ tất cả các microservices và cơ sở hạ tầng về một cơ sở dữ liệu duy nhất. Cơ chế này không chỉ giúp lập trình viên và quản trị viên hệ thống có thể dễ dàng tìm kiếm, truy vết chuỗi lỗi (Exception

Tracking) liên dịch vụ mà còn giúp theo dõi trạng thái sức khỏe hệ thống theo thời gian thực, phục vụ đắc lực cho công tác vận hành và bảo mật.

Thành phần cấu tạo của cụm EFK Stack (Elasticsearch - Fluentd - Kibana) Để xây dựng giải pháp quản lý log tập trung, dự án lựa chọn triển khai bộ công cụ EFK Stack (Elasticsearch - Fluentd - Kibana) theo mô hình non-intrusive (không can thiệp hay thay đổi mã nguồn của các dịch vụ nghiệp vụ). Bộ ba này được phân chia nhiệm vụ rõ ràng: Fluentd đóng vai trò là bộ thu thập và xử lý log (Log Collector), chạy dưới dạng container nền để đón dòng log xuất ra từ Docker Daemon; Elasticsearch đóng vai trò là lõi lưu trữ (Database NoSQL) và công cụ tìm kiếm phân tán mạnh mẽ, chịu trách nhiệm lưu trữ, phân tích cú pháp (parsing) và đánh chỉ mục (indexing) log với tốc độ phản hồi gần như tức thì; cuối cùng là Kibana đóng vai trò là giao diện trực quan hóa (Visualization Dashboard), cung cấp một Web UI trực quan giúp người vận hành thực hiện truy vấn log bằng ngôn ngữ KQL, lọc log theo từng container và vẽ biểu đồ giám sát sự cố.

Luồng dữ liệu và Nguyên lý hoạt động (Logging Data Flow) Dòng chảy dữ liệu log trong hệ thống được thiết lập tự động hóa hoàn toàn và khép kín qua các bước:

- Bước 1 (Ghi log): Khi các microservices (Java Spring Boot) thực hiện nghiệp vụ, log sẽ được in ra các luồng đầu ra tiêu chuẩn (stdout / stderr) của container.
- Bước 2 (Thu thập): Docker Daemon cấu hình sẵn Fluentd Logging Driver sẽ bắt các dòng log này một cách bất đồng bộ (fluentd-async: "true") nhằm tối ưu hiệu năng, tránh gây nghẽn dịch vụ gốc, sau đó đẩy về cổng 24224 của Fluentd.
- Bước 3 (Chuẩn hóa và Ghi nhận): Fluentd tiếp nhận logs, lọc bỏ logs rác, chuẩn hóa về định dạng JSON, ghi tạm vào vùng đệm (buffer file) để chống mất mát dữ liệu và tiến hành đẩy theo lô (batch) sang Elasticsearch (cổng 9200).
- Bước 4 (Đánh chỉ mục và Truy vấn): Elasticsearch tiếp nhận dữ liệu, tự động phân tích các trường dữ liệu (như container_name, @timestamp, log_level) và đánh chỉ mục theo ngày dạng bookland-YYYY.MM.DD. Người quản trị khi truy cập vào Kibana (cổng 5601) chỉ cần thiết lập Data View bookland-* để

truy vấn và giám sát trực quan toàn bộ nhật ký hoạt động của hệ thống thời gian thực.

2.6.5 Logging trong từng service

Mỗi service sử dụng SLF4J/Logback để ghi log trong các luồng nghiệp vụ chính, bao gồm log khi nhận request, khi gọi service khác, khi publish/consume Kafka message và khi phát sinh lỗi. Các log này được in ra stdout/stderr của container, sau đó được Docker Fluentd logging driver thu gom và chuyển về Fluentd. Nhờ vậy, nhóm phát triển vẫn giữ được cách ghi log đơn giản trong mã nguồn, đồng thời có thể phân tích tập trung trên Kibana thay vì phải kiểm tra thủ công từng container.

2.6.6 Observability trên Kubernetes và định hướng Distributed Tracing

Trong phạm vi triển khai hiện tại, hệ thống đã có observability ở mức logging trên Kubernetes thông qua Fluentd DaemonSet, Elasticsearch và Kibana. Fluentd thu thập log từ container/pod trên node Kubernetes, chuyển về Elasticsearch để đánh chỉ mục, sau đó Kibana cung cấp giao diện truy vấn và lọc log theo thời gian, service hoặc container. Cơ chế này giúp nhóm phát triển quan sát hệ thống tập trung hơn so với việc kiểm tra log thủ công từng pod.

Tuy nhiên, centralized logging trên Kubernetes chưa phải là distributed tracing theo nghĩa đầy đủ. Distributed tracing cần gắn và lan truyền traceId/spanId qua các service, đồng thời gửi span về các công cụ như OpenTelemetry Collector, Jaeger, Zipkin hoặc Grafana Tempo. Mã nguồn hiện tại chưa thể hiện rõ các dependency/cấu hình tracing này, vì vậy distributed tracing được xác định là hướng phát triển tiếp theo để liên kết một request xuyên suốt API Gateway, Order Service, Book Service, Event Service và Notification Service.

2.6.7 Triển khai trên Kubernetes

2.6.7.1. Tài nguyên triển khai Kubernetes

Hệ thống BookLand Microservice được đóng gói và vận hành trên nền tảng Kubernetes (sử dụng Minikube ở môi trường local hoặc các Managed Kubernetes Service như GKE/EKS ở môi trường Production). Kiến trúc này giúp tối ưu hóa khả năng co giãn (scaling), phục hồi tự động (self-healing), và tách biệt môi trường một cách an toàn. Triển khai của BookLand được phân tách thành 3 tầng tài nguyên chính:

Tầng 1: Hạ tầng & Dữ liệu lưu giữ (Stateful Infrastructure)

Do các cơ sở dữ liệu và message broker yêu cầu tính toàn vẹn dữ liệu cao và định danh mạng ổn định, tầng này sử dụng các tài nguyên K8s chuyên biệt:

- StatefulSet: Đảm bảo các Pod hạ tầng (như mysql-0, kafka-0) giữ nguyên tên và địa chỉ IP nội bộ cố định khi khởi động lại, phục vụ tốt cho việc clustering và replication.
- PersistentVolume (PV) & PersistentVolumeClaim (PVC): Gắn kết ổ đĩa vật lý của máy chủ vật lý/máy ảo vào container. Khi Pod database bị lỗi và khởi động lại, dữ liệu cũ vẫn được tự động mount ngược lại đúng Pod đó, chống mất mát dữ liệu.
- Service (ClusterIP): Tạo cổng kết nối nội bộ ổn định cho các microservice truy cập dữ liệu (ví dụ kết nối đến MySQL qua địa chỉ mysql:3306).

Tầng 2: Dịch vụ Ứng dụng (Stateless Microservices)

Các dịch vụ nghiệp vụ viết bằng Spring Boot và Node.js đều là dạng không lưu trạng thái (Stateless), cho phép co giãn nhanh chóng:

- Deployment: Định nghĩa cấu hình container (Docker Image, số lượng Replica, biến môi trường cấu hình cổng kết nối DB/Kafka). K8s sẽ tự động quản lý vòng đời và duy trì số lượng Pod hoạt động mong muốn.

- Service (ClusterIP): Đóng vai trò cân bằng tải nội bộ (Internal Load Balancer). Khi một service gọi service khác (ví dụ: order-service gọi book-service), nó chỉ cần gọi qua tên Service K8s `http://book-service:8083`. K8s sẽ tự động phân phối các request đều ra các Pod đang hoạt động bên dưới.
- ConfigMap & Secret: Tách biệt cấu hình môi trường ra khỏi mã nguồn ứng dụng, giúp dễ dàng thay đổi địa chỉ kết nối database hay khóa bí mật JWT mà không cần build lại Docker Image.

Tầng 3: Định tuyến Ngoại vi (Ingress Layer)

- Ingress Controller (Nginx Ingress): Đóng vai trò làm điểm tiếp nhận duy nhất cho toàn bộ traffic từ bên ngoài đi vào Cluster.
- Ingress Rules: Định nghĩa luật ánh xạ tên miền nghiệp vụ (như `api.bookland.local`) hoặc các đường dẫn URL trực tiếp tới API Gateway (`api-gateway:8080`). Nhờ đó, chúng ta ẩn giấu hoàn toàn các microservice khác phía sau Cluster, tăng cường tối đa tính bảo mật bảo vệ các dịch vụ nghiệp vụ nội bộ.

2.6.7.2. Các Nguyên lý Thiết kế Kubernetes Áp dụng trong Dự án

- Cách ly Tài nguyên qua Namespace (bookland): Toàn bộ tài nguyên ứng dụng, hạ tầng và cấu hình đều được đặt trong namespace riêng biệt tên là bookland. Điều này giúp cô lập hoàn toàn dự án với các dịch vụ mặc định của Kubernetes (như kube-system), tránh xung đột đặt tên tài nguyên và dễ dàng quản lý hạn mức tài nguyên (ResourceQuota).
- Cơ chế Tự phục hồi (Self-Healing): Sử dụng các cấu hình thăm dò sức khỏe (Liveness Probe và Readiness Probe) giúp Kubernetes tự động phát hiện các Pod bị treo, tràn bộ nhớ hay lỗi kết nối để hủy bỏ và tạo mới Pod thay thế ngay lập tức mà không cần sự can thiệp thủ công từ quản trị viên.
- Chiến lược Zero-Downtime Deployment (Rolling Update): Khi cập nhật phiên bản mới cho các Microservices, Kubernetes sẽ khởi động các Pod mới trước, kiểm tra sức khỏe thành công rồi mới tắt dần các Pod phiên bản cũ. Điều này

giúp hệ thống hoạt động liên tục 24/7 không bị gián đoạn dịch vụ khi tiến hành deploy code mới.

2.6.7 Dashboard giám sát — Kafka UI

Kafka UI (Provectus) được tích hợp tại port 8090, cung cấp dashboard web cho phép giám sát: danh sách topic, số lượng partition, message offset, consumer group và trạng thái consumer. Công cụ này hỗ trợ debug luồng event trong quá trình phát triển.

2.6.8 Swagger / OpenAPI Documentation

Mỗi microservice tích hợp SpringDoc OpenAPI 3.0, cung cấp trang Swagger UI tại `/swagger-ui/index.html`. Tất cả endpoint được document đầy đủ với mô tả, parameter, request/response schema và yêu cầu xác thực (BearerAuth).

2.6.9 Tích hợp thanh toán VNPay

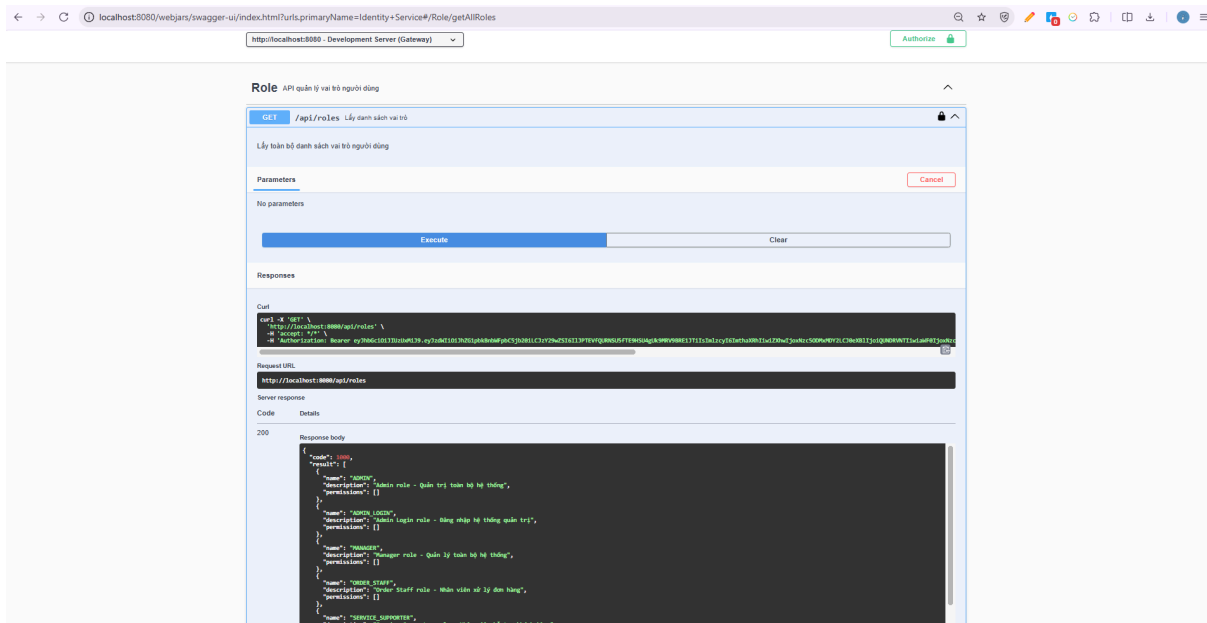
Order Service tích hợp VNPay — cổng thanh toán phổ biến tại Việt Nam. Luồng thanh toán: khách tạo đơn hàng → Order Service tạo URL thanh toán VNPay → khách thanh toán trên trang VNPay → VNPay callback IPN → Order Service xác nhận và cập nhật trạng thái đơn hàng.

CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG

3.1 Triển khai API Gateway

API Gateway là điểm truy cập duy nhất của hệ thống backend. Thành phần này chịu trách nhiệm định tuyến request đến các microservice, xử lý CORS, kiểm tra các endpoint công khai và xác thực các endpoint yêu cầu token. Khi client gửi request đến các tài nguyên được bảo vệ, Gateway lấy JWT trong header Authorization, kiểm tra token thông qua Identity Service và chuyển tiếp thông tin người dùng xuống các service phía sau bằng các header nội bộ.

Trong dự án, API Gateway được triển khai bằng Spring Cloud Gateway và chạy tại cổng 8080. Các route được cấu hình để chuyển request đến Identity Service, User Service, Book Service, Order Service, Event Service, Notification Service, File Service và Search Service. Việc đặt API Gateway ở phía trước giúp client không cần biết trực tiếp địa chỉ của từng service, đồng thời tạo ra một lớp kiểm soát thống nhất cho xác thực và định tuyến.



Hình 3.1 API Doc Swagger

3.2. Triển khai các microservice

3.2.1. Triển khai bằng Docker Compose

Trong môi trường phát triển cục bộ, hệ thống được triển khai bằng Docker Compose. Cách triển khai này giúp khởi chạy đồng thời nhiều service và các thành phần hạ tầng phụ thuộc mà không cần cài đặt thủ công từng thành phần trên máy chủ. Trước khi chạy Docker Compose, các service Java được build thành file JAR bằng Maven. Sau đó, Dockerfile của từng service sao chép file JAR vào container và khởi động ứng dụng bằng môi trường Java 17.

Quy trình triển khai cơ bản gồm hai bước chính:

Bước 1: build toàn bộ mã nguồn backend bằng lệnh *mvn clean package -DskipTests* tại thư mục gốc của dự án.

Bước 2: khởi chạy hệ thống bằng lệnh *docker compose up -d --build*.

Sau khi hoàn tất, các container nghiệp vụ và container hạ tầng được tạo trong cùng một network nội bộ của Docker Compose, nhờ đó các service có thể gọi nhau bằng tên service như book-service, order-service hoặc notification-service.

File docker-compose.yml của dự án khai báo đầy đủ các service chính, bao gồm API Gateway, Identity Service, User Service, Book Service, Order Service, Event Service, Notification Service, File Service, Search Service, Chat Service và Frontend. Ngoài ra, file này cũng khai báo các thành phần hạ tầng như MySQL cho từng service nghiệp vụ, MongoDB, Redis, Kafka, Zookeeper, Kafka UI, Elasticsearch, Fluentd, Kibana và MinIO. Mỗi thành phần có cổng ánh xạ ra máy host để phục vụ kiểm thử, giám sát và truy cập trong quá trình phát triển.

Kết quả: Toàn bộ hệ thống được containerize với Docker Compose

```
PS C:\Users\kaita> docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
6c92bb38599c	p_bookland_ms-frontend	bookland-frontend	"/docker-entrypoint.s..."	55 minutes ago	Up 31 seconds	0.0.0.0:5173->80
140c1c1b9e2d	p_bookland_ms-chat-service	chat-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8089->80
89/tcp, [::]:8089->8089/tcp						
1ef5c8c58a01	p_bookland_ms-notification-service	notification-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8086->80
86/tcp, [::]:8086->8086/tcp						
f5ce784f7065	p_bookland_ms-user-service	user-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8082->80
82/tcp, [::]:8082->8082/tcp						
4f35df2da839	p_bookland_ms-search-service	search-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8088->80
88/tcp, [::]:8088->8088/tcp						
1381877cfba6	p_bookland_ms-book-service	book-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8083->80
83/tcp, [::]:8083->8083/tcp						
b33faa58d9ef	p_bookland_ms-order-service	order-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8084->80
84/tcp, [::]:8084->8084/tcp						
212a0a3271b1	p_bookland_ms-event-service	event-service	"java -jar app.jar"	2 hours ago	Up 31 seconds	0.0.0.0:8085->80
85/tcp, [::]:8085->8085/tcp						
4f6c444b7719	p_bookland_ms-api-gateway	api-gateway	"java -jar app.jar"	2 hours ago	Up 32 seconds	0.0.0.0:8080->80
80/tcp, [::]:8080->8080/tcp						
54851c0acd7c	p_bookland_ms-identity-service	identity-service	"java -jar app.jar"	2 hours ago	Up 30 seconds	0.0.0.0:8081->80
81/tcp, [::]:8081->8081/tcp						
f943573fdec	p_bookland_ms-file-service	file-service	"java -jar app.jar"	2 hours ago	Up 32 seconds	0.0.0.0:8087->80
87/tcp, [::]:8087->8087/tcp						
cc9de27dbe2d	confluentinc/cp-kafka:7.5.0	bookland-kafka	"/etc/confluent/dock..."	2 hours ago	Up 31 seconds	0.0.0.0:9092->90
92/tcp, [::]:9092->9092/tcp						
64158b93377d	mongo:7.0	notification-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:27017->2
7017/tcp, [::]:27017->27017/tcp						
81a974a14752	mysql:8.0	chat-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3312->33
06/tcp, [::]:3312->3306/tcp						
4cf1bf6a2546	mysql:8.0	order-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3310->33
06/tcp, [::]:3310->3306/tcp						
e957b892e093	mysql:8.0	user-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3308->33
06/tcp, [::]:3308->3306/tcp						
0c781c48864b	redis:7-alpine	bookland-redis	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:6379->63
79/tcp, [::]:6379->6379/tcp						
e3b6f3cc2509	mysql:8.0	event-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3311->33
06/tcp, [::]:3311->3306/tcp						
d66bb503c780	provectuslabs/kafka-ui:latest	kafka-ui	"/bin/sh -c 'java --..."	2 hours ago	Up 33 seconds	0.0.0.0:8090->80
80/tcp, [::]:8090->8090/tcp						
d94b9eb9a40a	minio/minio:latest	bookland-minio	"/usr/bin/docker-ent..."	2 hours ago	Up 33 seconds	0.0.0.0:9000-900
1->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp						
d13b7ba70ff3	mysql:8.0	identity-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3307->33
06/tcp, [::]:3307->3306/tcp						
55d897afe82a	mysql:8.0	book-db	"docker-entrypoint.s..."	2 hours ago	Up 33 seconds	0.0.0.0:3309->33
06/tcp, [::]:3309->3306/tcp						
fbae85193043	confluentinc/cp-zookeeper:7.5.0	zookeeper	"/etc/confluent/dock..."	2 hours ago	Up 33 seconds	2181/tcp, 2888/t
cp, 3888/tcp						
acd8d62be9fe	docker.elastic.co/elasticsearch/elasticsearch:8.11.0	bookland-elasticsearch	"/bin/tini -- /usr/l..."	2 hours ago	Up 33 seconds	0.0.0.0:9200->92
00/tcp, [::]:9200->9200/tcp						

Hình 3.2 Các container hệ thống

3.2.2. Triển khai bằng Kubernetes

Bên cạnh Docker Compose, dự án cũng chuẩn bị các manifest Kubernetes trong thư mục k8s nhằm mô phỏng cách triển khai hệ thống trong môi trường gần với production hơn. Các manifest được chia thành ba nhóm chính.

- Nhóm 01 - infrastructure mô tả các thành phần hạ tầng như MySQL, MongoDB, Kafka, Redis, Elasticsearch, Fluentd, Kibana và MinIO.

- Nhóm 02 - services mô tả Deployment và Service cho các microservice nghiệp vụ.
- Nhóm 03 - ingress mô tả cấu hình Ingress để đưa API Gateway ra ngoài cụm Kubernetes.

Trong Kubernetes, mỗi microservice được triển khai dưới dạng Deployment, còn khả năng truy cập nội bộ được cung cấp thông qua tài nguyên Service. Cơ chế DNS nội bộ của Kubernetes cho phép các service gọi nhau bằng tên ổn định, ví dụ `http://book-service:8083` hoặc `http://order-service:8084`. Với cách tiếp cận này, hệ thống có thể tăng số lượng replica của từng service mà không làm thay đổi cách các thành phần khác truy cập đến service đó.

```
k8s-user@kaitams-1:~/P_BookLand_MS$ kubectl get pods -n bookland -w
```

NAME	READY	STATUS	RESTARTS	AGE
api-gateway-cbf9769df-tc5vq	1/1	Running	0	11m
book-service-67bd5fd964-4mdn6	1/1	Running	0	11m
chat-service-894f4fbdb-lv6cn	1/1	Running	0	11m
elasticsearch-0	1/1	Running	0	11m
event-service-66bdc997fc-gdfc8	1/1	Running	0	11m
file-service-57d5f8569d-rqcqf	1/1	Running	0	19s
identity-service-65b4785cc4-x4gxq	1/1	Running	0	18s
kafka-0	1/1	Running	0	11m
minio-0	1/1	Running	0	11m
mongodb-0	1/1	Running	0	11m
mysql-0	1/1	Running	0	11m
notification-service-8867b6954-64m99	1/1	Running	0	18s
order-service-6884f4448c-kcqbz	1/1	Running	0	18s
redis-65b6d5979b-2vq75	1/1	Running	0	11m
search-service-678b495fc6-6jx7t	1/1	Running	1 (9m55s ago)	11m
user-service-8f9fcd5bb-jvhk5	1/1	Running	0	11m

Hình3.3 kubectl get pods

```
k8s-user@kaitams-1:~/P_BookLand_MS$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	28m

Hình 3.4 kubectl get services

3.3. Triển khai Kafka và giao tiếp bất đồng bộ

Kafka được sử dụng làm nền tảng giao tiếp bất đồng bộ giữa các service. Trong môi trường Docker Compose, Kafka chạy cùng Zookeeper và được cấu hình để các service nội bộ truy cập thông qua bootstrap server bookland-kafka:29092. Kafka UI được triển khai tại cổng 8090, hỗ trợ quan sát topic, message, offset và trạng thái consumer group trong quá trình kiểm thử.

Luồng bất đồng bộ tiêu biểu của hệ thống là luồng gửi thông báo sau khi phát sinh sự kiện đơn hàng. Order Service đóng vai trò producer và gửi message vào topic notification-events. Notification Service đóng vai trò consumer, nhận message từ Kafka, tạo thông báo trong MongoDB, đẩy thông báo realtime qua WebSocket và gửi email nếu sự kiện yêu cầu. Luồng này giúp Order Service không bị phụ thuộc trực tiếp vào thời gian xử lý email hoặc WebSocket của Notification Service.

Trong trường hợp xử lý message gặp lỗi, Notification Consumer hiện đã bao bọc logic xử lý bằng cơ chế bắt ngoại lệ và ghi log lỗi. Cách xử lý này bảo đảm lỗi của một message không làm dừng toàn bộ tiến trình consumer. Tuy nhiên, hệ thống hiện chưa cấu hình retry topic hoặc dead-letter topic chuyên biệt; đây là điểm có thể tiếp tục mở rộng ở các phiên bản sau.

3.4. Triển khai Elasticsearch và Redis lưu trữ file

Elasticsearch được sử dụng để phục vụ chức năng tìm kiếm sách. Search Service lưu trữ dữ liệu tìm kiếm dưới dạng document trong Elasticsearch và cung cấp API tìm kiếm theo từ khóa, tác giả, danh mục, khoảng giá và phân trang. Khi Search Service hoặc Elasticsearch tạm thời không phản hồi trong một số luồng tìm kiếm qua Book Service, Book Service có cơ chế fallback sang truy vấn trực tiếp cơ sở dữ liệu bằng JPA Specification. Cơ chế này giúp chức năng xem danh sách sách vẫn có thể hoạt động ở mức cơ bản dù thành phần tìm kiếm nâng cao gặp sự cố.

Redis được triển khai như một thành phần hỗ trợ cho các nhu cầu liên quan đến xác thực, cache hoặc blacklist token tùy theo từng service. Trong kiến trúc

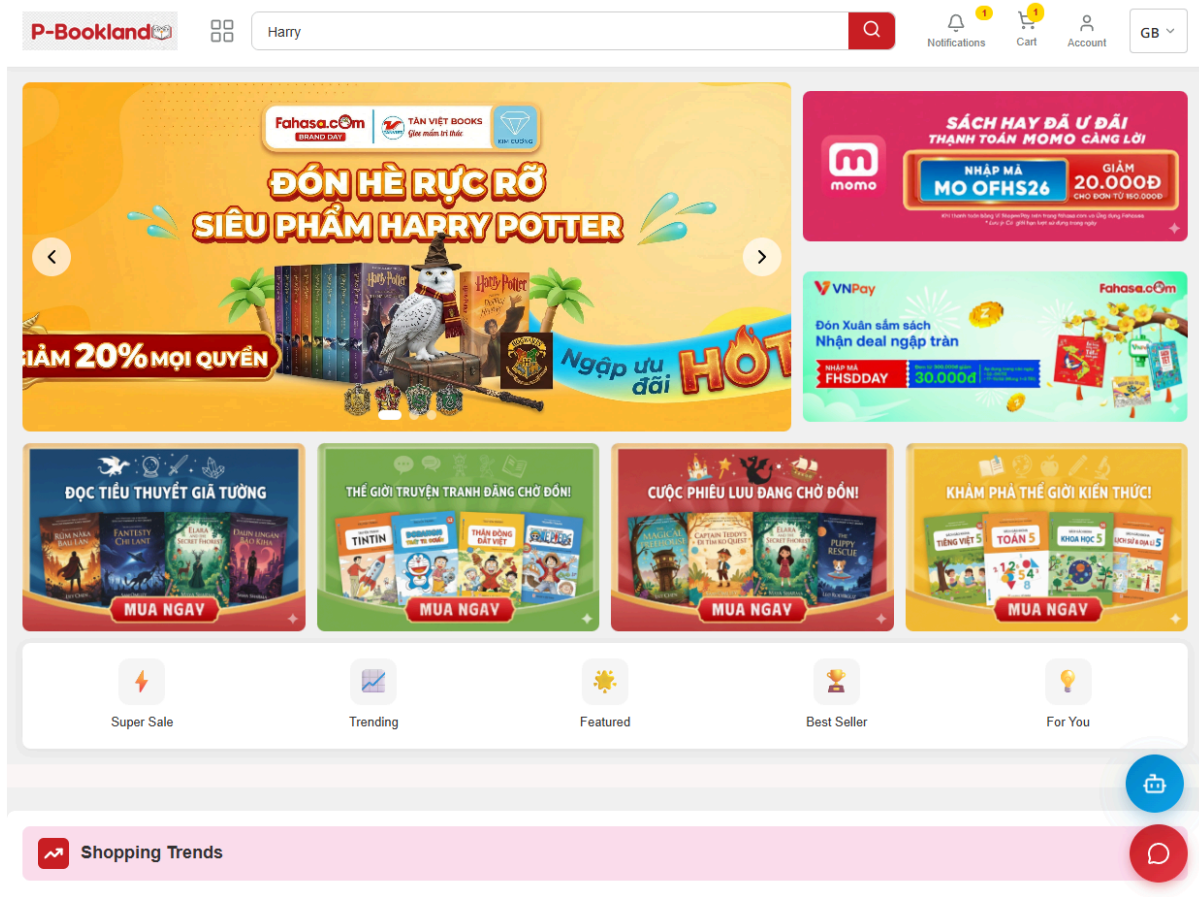
microservice, Redis giúp giảm chi phí truy cập lặp lại đối với những dữ liệu ngắn hạn và có tần suất sử dụng cao.

File Service được thiết kế để tách riêng nghiệp vụ upload và lưu trữ ảnh khỏi các service còn lại. Trong mã nguồn hiện tại, UploadController sử dụng SupabaseStorageService để upload, liệt kê và xóa ảnh trên Supabase Storage. Bên cạnh đó, Docker Compose vẫn triển khai MinIO như một object storage tương thích S3 để phục vụ kịch bản self-hosted hoặc thay thế backend lưu trữ khi cần. Các service khác chỉ cần lưu URL hoặc định danh tệp, còn thao tác upload được tập trung tại File Service.

3.6. Giao diện frontend

Phần giao diện người dùng của BookLand được xây dựng bằng React, Vite và TypeScript. Frontend giao tiếp với backend thông qua API Gateway, nhờ đó địa chỉ truy cập của các microservice được che giấu khỏi phía client. Giao diện bao gồm các nhóm màn hình chính như trang chủ, danh sách sách, chi tiết sách, giỏ hàng, thanh toán, đơn hàng cá nhân, hồ sơ người dùng và các trang quản trị.

Trong môi trường phát triển, frontend được chạy bằng npm run dev tại thư mục BookLand_FE. Các request HTTP được cấu hình thông qua lớp API client, trong đó token đăng nhập được gắn vào header để truy cập các tài nguyên cần xác thực. Với Notification Service, frontend có thể nhận thông báo realtime qua WebSocket khi người dùng phát sinh các sự kiện liên quan đến đơn hàng hoặc hệ thống.



Hình 3.5 Trang chủ

P-Bookland

Harry

Notifications

Cart

Account

GB

Home > Books

Sort by: Default

12 products

Search results for: "Harry"

Clear All Filters

PRICE RANGE

☐ 0đ - 150,000đ
 ☐ 150,000đ - 300,000đ
 ☐ 300,000đ - 500,000đ
 ☐ 500,000đ - 700,000đ
 ☐ 700,000đ - Up

CATEGORIES

Search categories...

Try:

Comics

Horror

Romance

Science

AUTHORS

Search authors...

Try:

Nouven Nhat Anh

To Hoai

Harry Potter và Bảo bối Tử thần

153.000 đ -10%

470.000 đ

★★★★★

Harry Potter và Hoàng tử lai

139.500 đ -10%

455.000 đ

★★★★★

Harry Potter và Hội Phượng Hoàng

144.000 đ -10%

460.000 đ

★★★★★

Harry Potter và Chiếc cốc lửa

135.000 đ -10%

450.000 đ

★★★★★

Harry Potter và Tên tù nhân ngục Azkaban

121.500 đ -10%

Harry Potter và Phòng chứa Bí mật

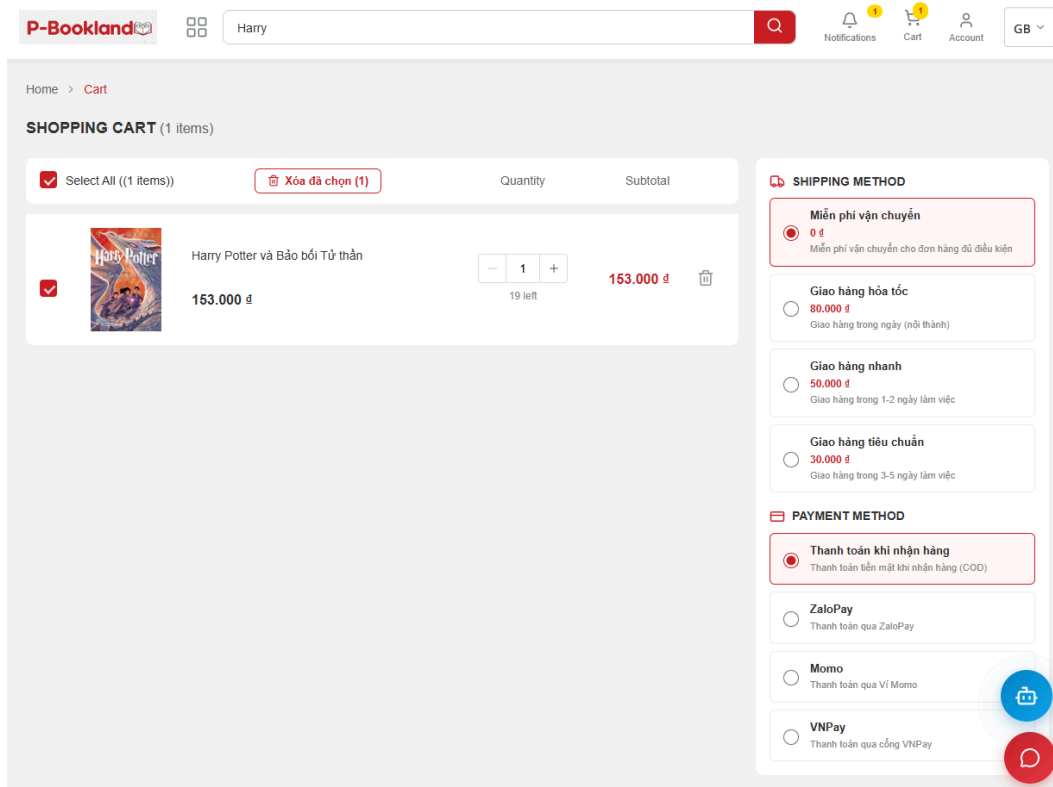
117.000 đ -10%

Harry Potter và Hòn đá Phù thủy

108.000 đ -10%

Hình 3.6 Danh sách sản phẩm

53



Hình 3.7 Giỏ hàng

CÔNG THANH TOÁN VNPay Giao dịch hết hạn sau **14 : 43**

Thông tin đơn hàng (Test)

Số tiền thanh toán **229.500 VND**

Giá trị đơn hàng **229.500 VND**

Phí giao dịch **0 VND**

Mã đơn hàng **02109233**

Nhà cung cấp **https://vnshop.vn/**

Thanh toán qua Ngân hàng NCB

Thẻ nội địa

Số thẻ *******2198**

Tên chủ thẻ **NGUYEN VAN A**

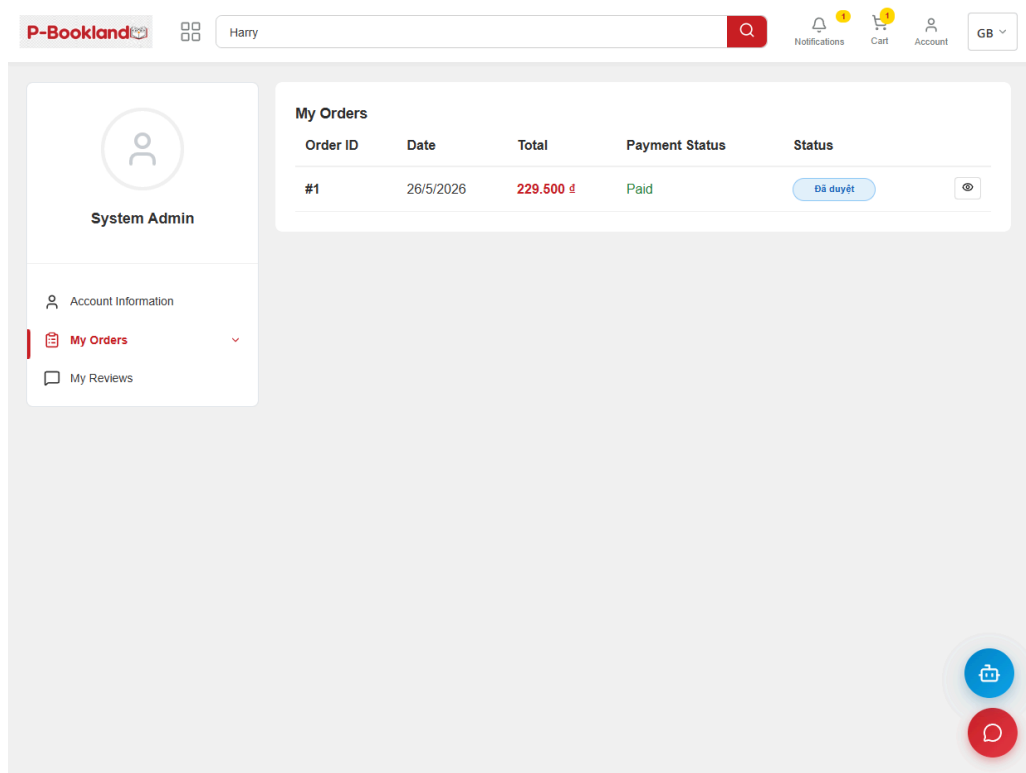
Ngày phát hành **07/15**

Mã khuyến mại **Chọn hoặc nhập mã**

[Điều kiện sử dụng dịch vụ](#)

Hủy thanh toán **Tiếp tục**

hotrovnpay@vnpay.vn



Hình 3.8 Quản lý hóa đơn đã mua

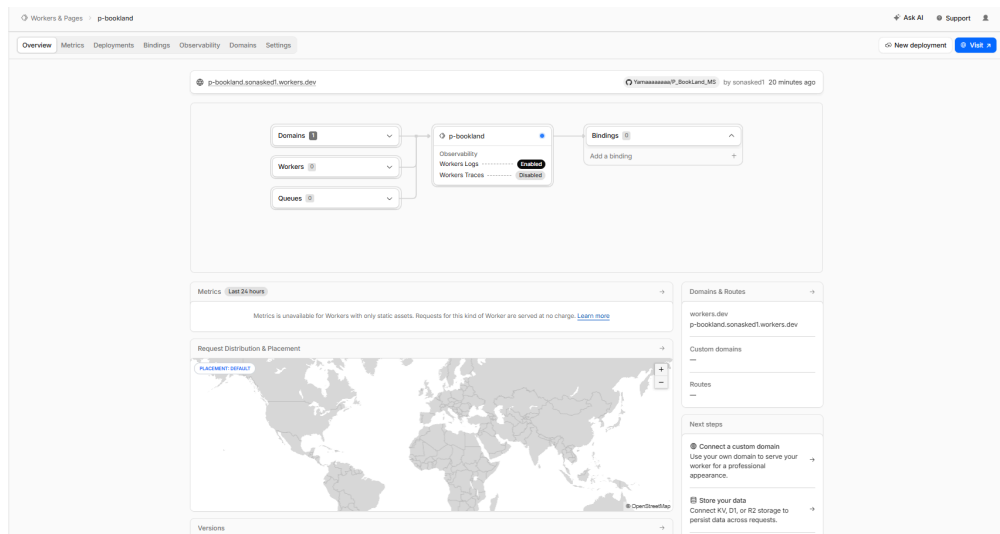
3.7. Triển khai Deploy

Sau khi hoàn thiện môi trường chạy cục bộ, dự án được đóng gói để triển khai theo hai phần độc lập: frontend được deploy riêng trên Cloudflare Worker, còn backend và các thành phần hạ tầng được deploy trên máy chủ Kamatera Cloud thông qua Kubernetes. Cách tách này phù hợp với kiến trúc microservices vì frontend có thể được phân phối gần người dùng hơn, trong khi backend vẫn được quản lý trong cụm container riêng.

3.7.1. Deploy FE với Cloudflare Worker

Frontend BookLand được build từ mã nguồn React/Vite thành static assets và đưa lên Cloudflare Worker để phục vụ giao diện người dùng. Các request API từ frontend không gọi trực tiếp từng microservice mà đi qua API Gateway, giúp che giấu địa chỉ nội bộ của hệ thống backend và giảm coupling giữa giao diện với từng service

ng nghiệp vụ. Khi cần thay đổi domain backend hoặc môi trường triển khai, frontend chỉ cần cập nhật cấu hình endpoint của API Gateway.

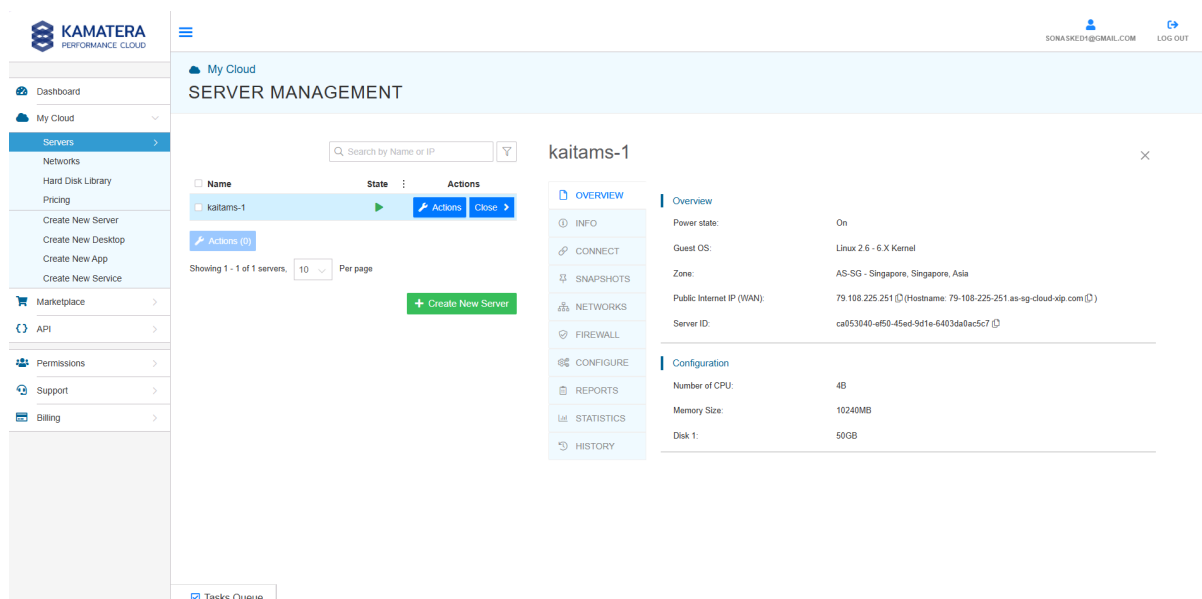


Hình 3.9 Cloudflare Worker

3.7.2. Deploy BE với Kamatera Cloud và Kubernetes

Backend được triển khai trên máy chủ Kamatera Cloud dưới dạng cụm Kubernetes. Các image của microservice được build từ Dockerfile riêng trong từng service, sau đó được khai báo bằng các manifest trong thư mục k8s. API Gateway là điểm vào chính của backend; Ingress ánh xạ domain `api.p-bookland.io.vn` và `api.bookland.local` về service `api-gateway` cổng 8080. Các service nghiệp vụ còn lại chỉ được truy cập trong mạng nội bộ Kubernetes thông qua Service DNS như `book-service`, `order-service` hoặc `notification-service`.

Các thành phần có trạng thái như MySQL, MongoDB, Kafka, Redis, Elasticsearch và MinIO được triển khai trong nhóm infrastructure. Các microservice nghiệp vụ được triển khai trong nhóm services dưới dạng Deployment/Service. Cách tổ chức này giúp tách rõ hạ tầng dùng chung với logic nghiệp vụ, thuận tiện cho việc cập nhật service, kiểm tra pod, scale replica và xử lý sự cố trong quá trình vận hành.



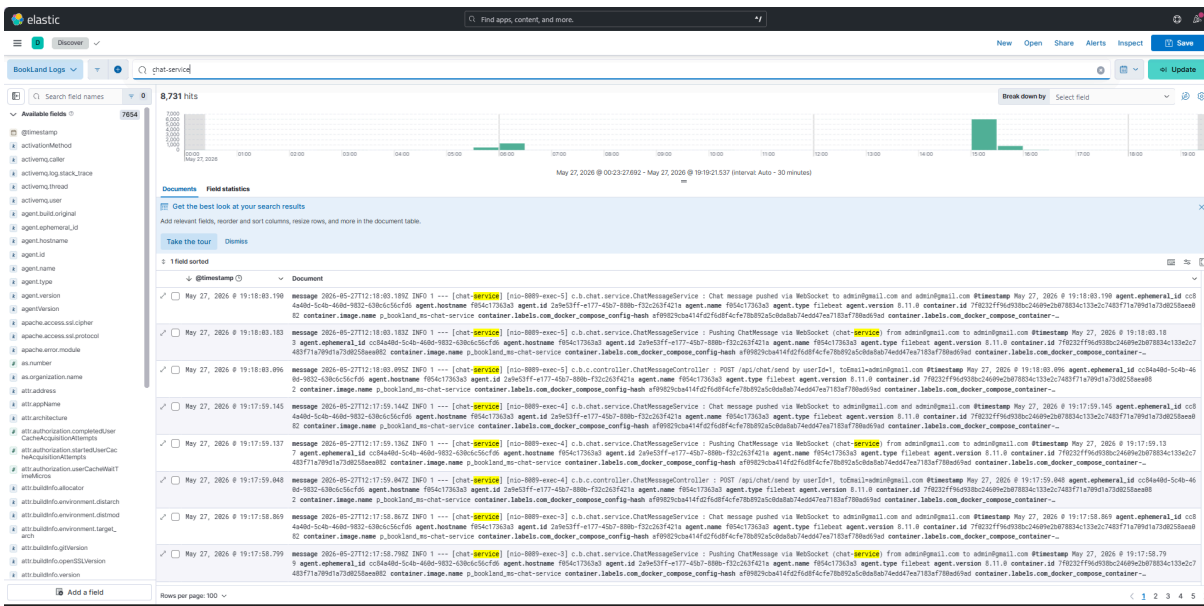
Hình 3.10 Kamatera Cloud

3.8. Centralize Logging

Hệ thống đã tích hợp centralized logging bằng EFK Stack gồm Elasticsearch, Fluentd và Kibana. Trong Docker Compose, mỗi container nghiệp vụ cấu hình logging driver là fluentd, gửi log bất đồng bộ về địa chỉ tcp://127.0.0.1:24224 với tag dạng docker.{{.Name}}. Fluentd nhận log qua forward input, ghi buffer file tại /fluentd/log để giảm rủi ro mất log, sau đó đẩy log sang Elasticsearch với index theo ngày dạng bookland-YYYY.MM.DD. Kibana kết nối tới Elasticsearch tại cổng 5601 và sử dụng Data View bookland-* để truy vấn, lọc, theo dõi log theo container và thời gian.

Trong Kubernetes, Fluentd được triển khai dưới dạng DaemonSet kèm ServiceAccount, ClusterRole và ClusterRoleBinding để có quyền đọc log container trên các node. Fluentd mount /var/log và /var/lib/docker/containers từ host, loại trừ log của chính Fluentd, Elasticsearch và Kibana để tránh vòng lặp log, rồi chuyển dữ liệu về Elasticsearch service trong namespace bookland. Kibana được triển khai bằng

Deployment và Service bookland-kibana, có thể truy cập thông qua port-forward khi cần quan sát log trong môi trường cloud.



Hình 3.11 Kibana UI

CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường thử nghiệm

Quá trình thử nghiệm được thực hiện trong môi trường cục bộ với Docker Compose. Các service backend được build bằng Maven và chạy trong container. Frontend được chạy riêng bằng Vite để thuận tiện cho việc phát triển và quan sát giao diện. Các API được kiểm thử bằng trình duyệt, Swagger UI, Postman, Kafka UI và Kibana/Fluentd log.

Hệ điều hành: Windows 11

Backend: Spring Boot 3, Java 17, Maven

Frontend: React, Vite, TypeScript, Node.js

Container: Docker Desktop, Docker Compose v2

Cơ sở dữ liệu: MySQL 8, MongoDB 7

Hạ tầng hỗ trợ: Kafka, Zookeeper, Redis, Elasticsearch, MinIO

Công cụ kiểm thử: Swagger UI, Postman, Kafka UI, Docker logs

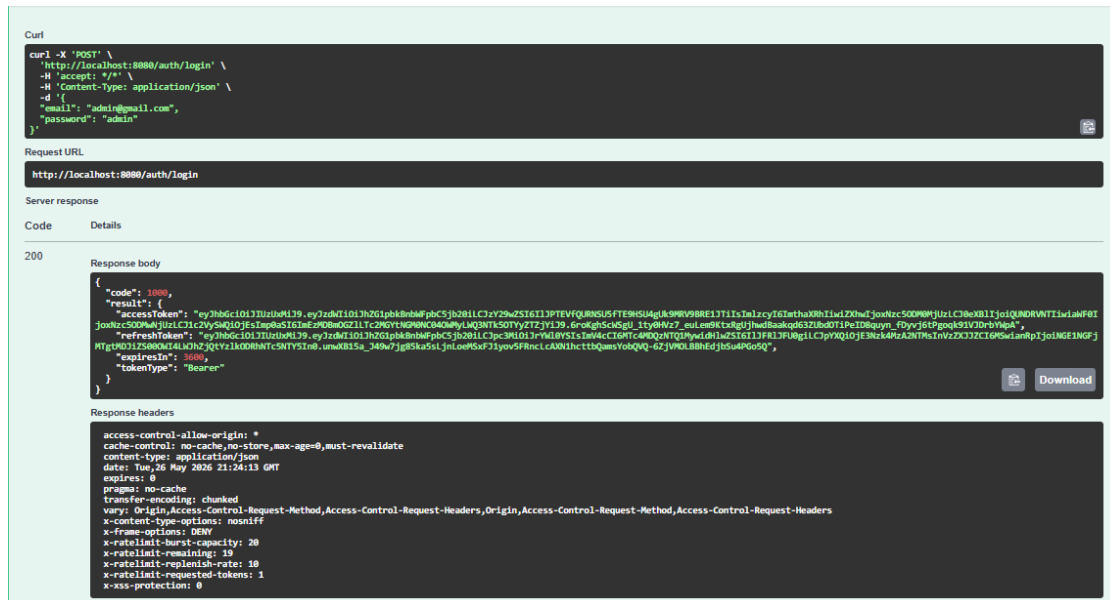
4.2 Kiểm thử các nghiệp vụ chính

4.2.1 Xác thực và phân quyền

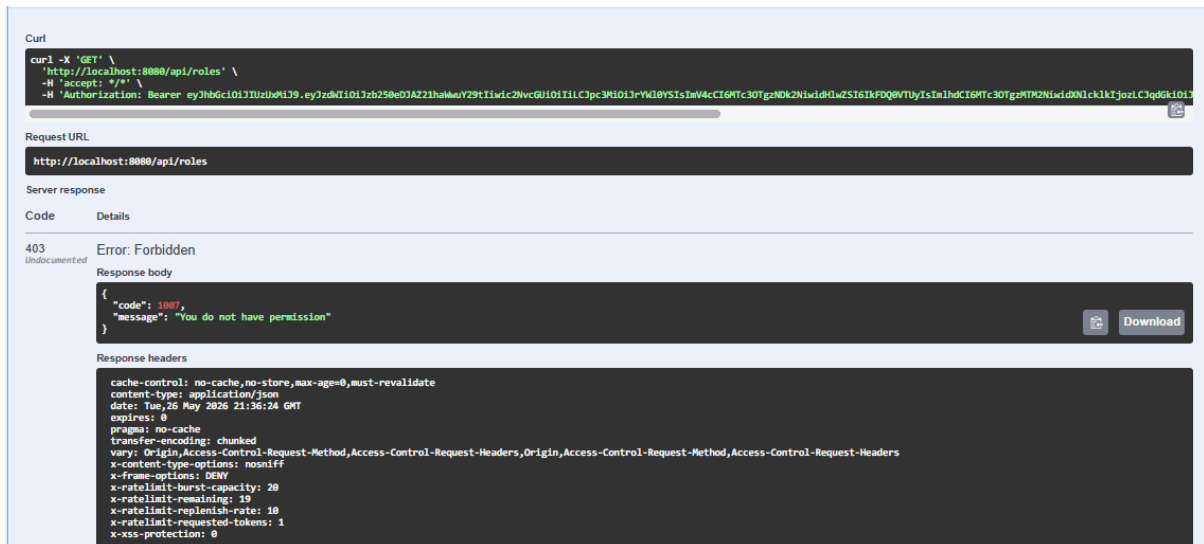
Nhóm tiên hành kiểm thử luồng đăng ký, đăng nhập và truy cập tài nguyên được bảo vệ. Khi người dùng đăng ký, Identity Service tạo tài khoản trong identity_db và có thể phối hợp với User Service để tạo hồ sơ người dùng. Khi người dùng đăng nhập thành công, hệ thống trả về JWT để client sử dụng cho các request tiếp theo.

Đối với các endpoint yêu cầu xác thực, request không có token hoặc token không hợp lệ bị API Gateway từ chối. Request có token hợp lệ được Gateway chuyển tiếp đến service đích. Kết quả kiểm thử cho thấy cơ chế xác thực tập trung tại

Gateway giúp giảm lặp lại logic kiểm tra token ở phía frontend và tạo ra một lớp bảo vệ thống nhất cho các service phía sau.



Hình 4.1 Login



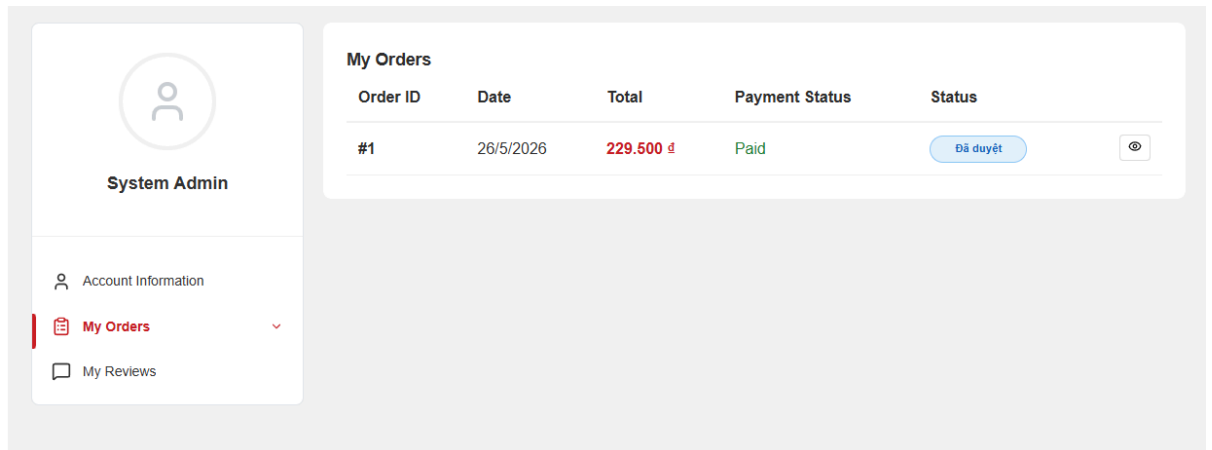
4.2.2 Luồng giỏ hàng, đặt hàng và Kafka notification

Sau khi đơn hàng được tạo hoặc cập nhật, Order Service gửi sự kiện thông báo đến Kafka. Notification Service lắng nghe topic notification-events, nhận message, tạo thông báo trong MongoDB và gửi thông báo real-time qua WebSocket. Nếu sự kiện có yêu cầu gửi email, service tiếp tục xử lý email dựa trên thông tin người nhận. Cách tổ chức này giúp xử lý thông báo tách khỏi luồng đặt hàng chính, từ đó giảm độ phụ thuộc giữa Order Service và Notification Service.

Hình 4.7 Thanh toán

64

Hình 4.8 Thanh toán thành công

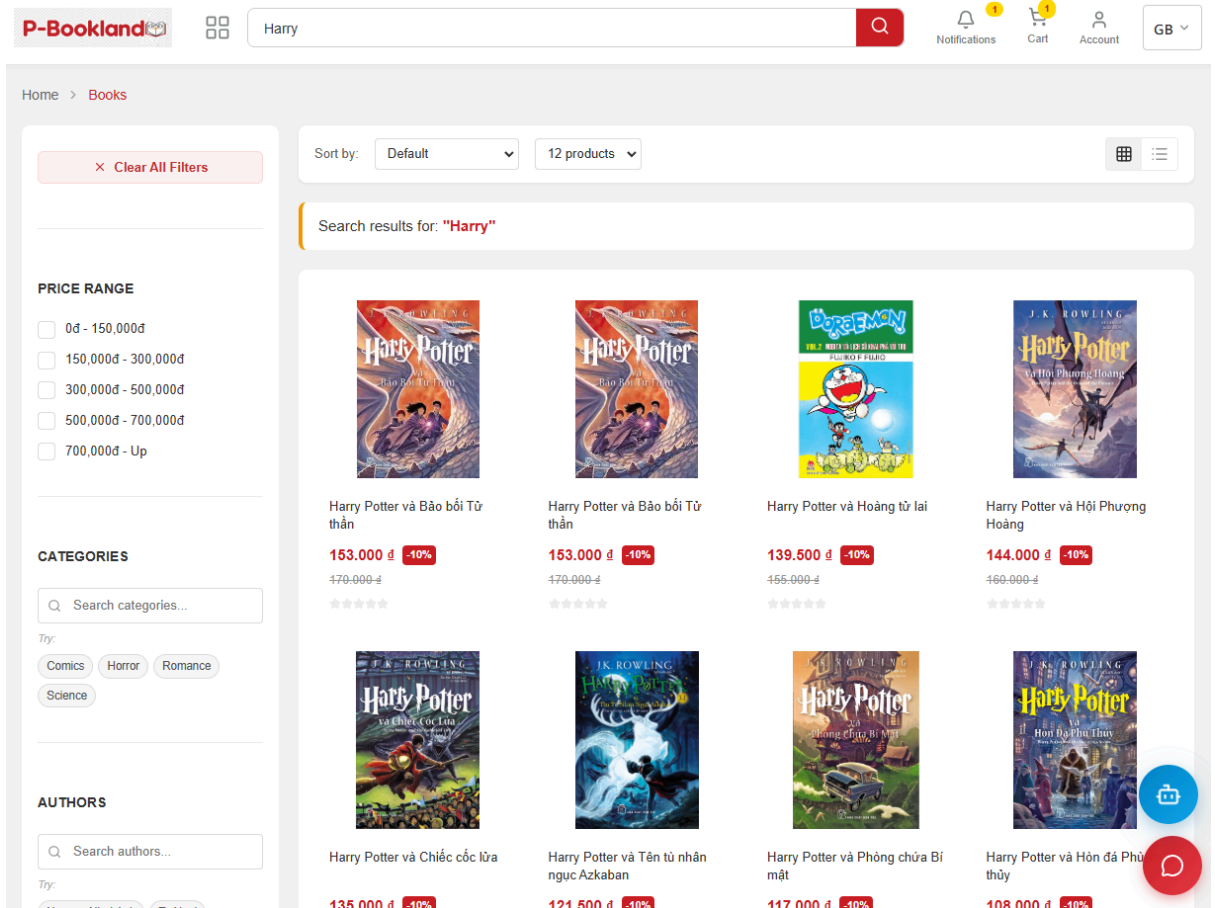


Hình 4.9 Hóa đơn đã thanh toán

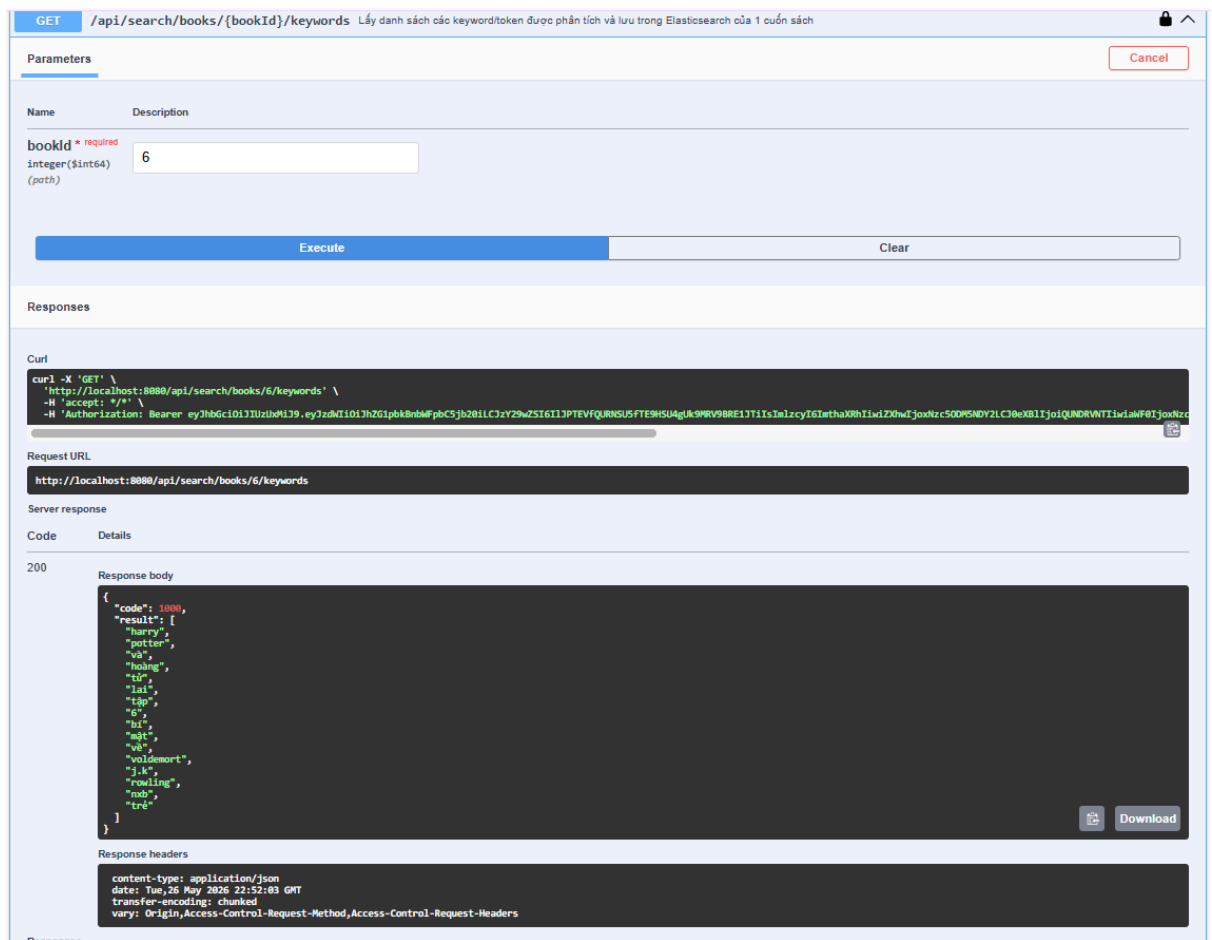
4.2.3 Luồng quản lý sách và tìm kiếm

Luồng quản lý sách được kiểm thử thông qua các thao tác thêm, sửa, xóa và truy vấn sách. Book Service lưu dữ liệu sách, tác giả, danh mục, nhà xuất bản và series trong book_db. Khi người dùng truy cập danh sách sách, hệ thống có thể sử dụng Search Service để tìm kiếm nâng cao hoặc truy vấn trực tiếp dữ liệu quan hệ trong trường hợp cần thiết.

Đối với chức năng tìm kiếm, Search Service truy vấn dữ liệu trong Elasticsearch và trả về kết quả theo từ khóa, bộ lọc và phân trang. Một điểm đáng chú ý là Book Service đã có cơ chế dự phòng khi Search Service gặp lỗi: service ghi log lỗi và chuyển sang truy vấn cơ sở dữ liệu bằng Specification. Kết quả này cho thấy hệ thống đã có mức chịu lỗi cơ bản đối với thành phần tìm kiếm.



Hình 4.10 Tìm kiếm



Hình 4.11 Dữ liệu trong Elastic DB

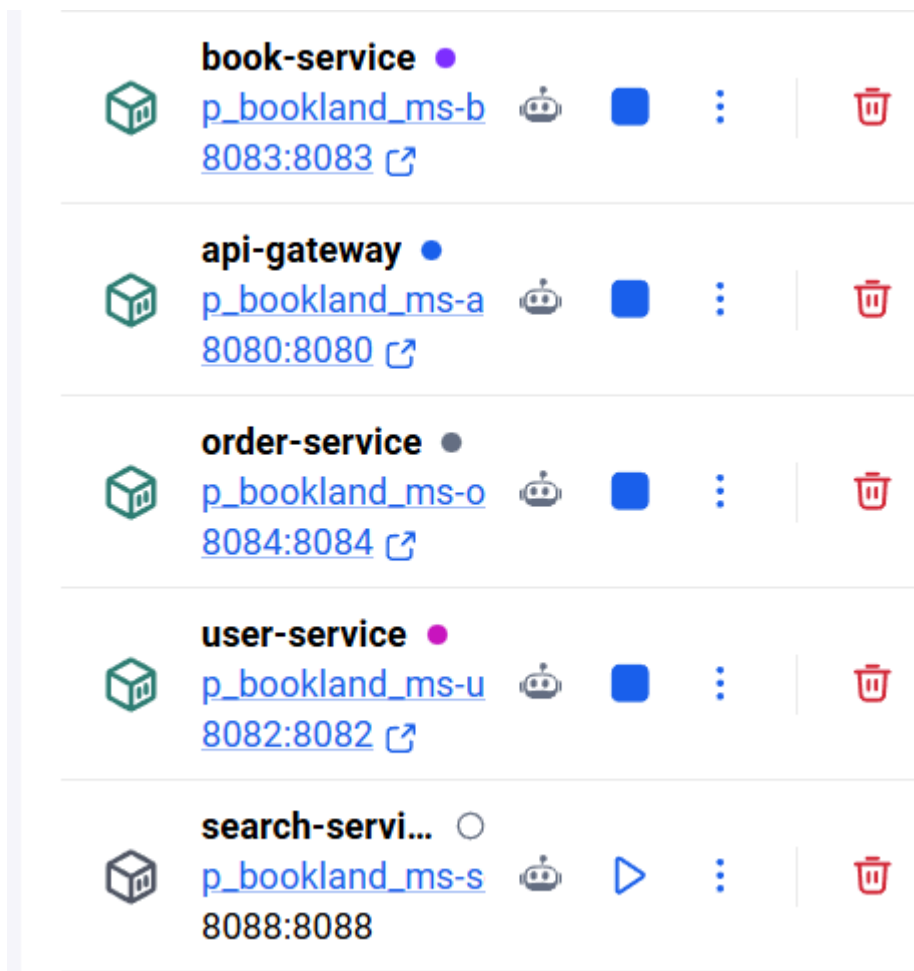
4.2.4 Kiểm thử xử lý lỗi cơ bản

Trong môi trường phân tán, lỗi có thể phát sinh do service tạm thời không phản hồi, dữ liệu đầu vào không hợp lệ hoặc message bất đồng bộ xử lý thất bại. Hệ thống BookLand đã triển khai xử lý lỗi ở mức cơ bản thông qua các lớp GlobalExceptionHandler trong từng service. Các ngoại lệ nghiệp vụ được ánh xạ về mã lỗi và thông báo rõ ràng, còn các lỗi không dự kiến được ghi log để phục vụ quá trình phân tích.

Khi một service gọi service khác bằng Feign hoặc WebClient và gặp lỗi, một số luồng đã có cơ chế bắt ngoại lệ và ghi log. Ví dụ, Book Service ghi nhận lỗi khi không gọi được Search Service và chuyển sang truy vấn database; Order Service ghi log khi không lấy được thông tin sách hoặc không cập nhật được tồn kho; Notification

Service ghi log khi không lấy được email người dùng hoặc khi không gửi được WebSocket notification. Đối với Kafka consumer, lỗi trong quá trình xử lý message được bắt và ghi log để tránh làm dừng consumer.

Ngoài xử lý ngoại lệ cơ bản, hệ thống đã có fallback ở một số luồng như tìm kiếm sách, giúp chức năng cốt lõi vẫn có thể hoạt động khi Search Service hoặc Elasticsearch tạm thời không phản hồi. API Gateway cũng đã chuẩn bị dependency/cấu hình nền Resilience4j, nhưng circuit breaker/retry policy chưa được áp dụng hoàn chỉnh cho toàn bộ route hoặc Feign Client. Với Kafka, hệ thống vẫn có thể tiếp tục bổ sung retry topic và dead-letter topic chuyên biệt để kiểm soát tốt hơn các message xử lý thất bại.

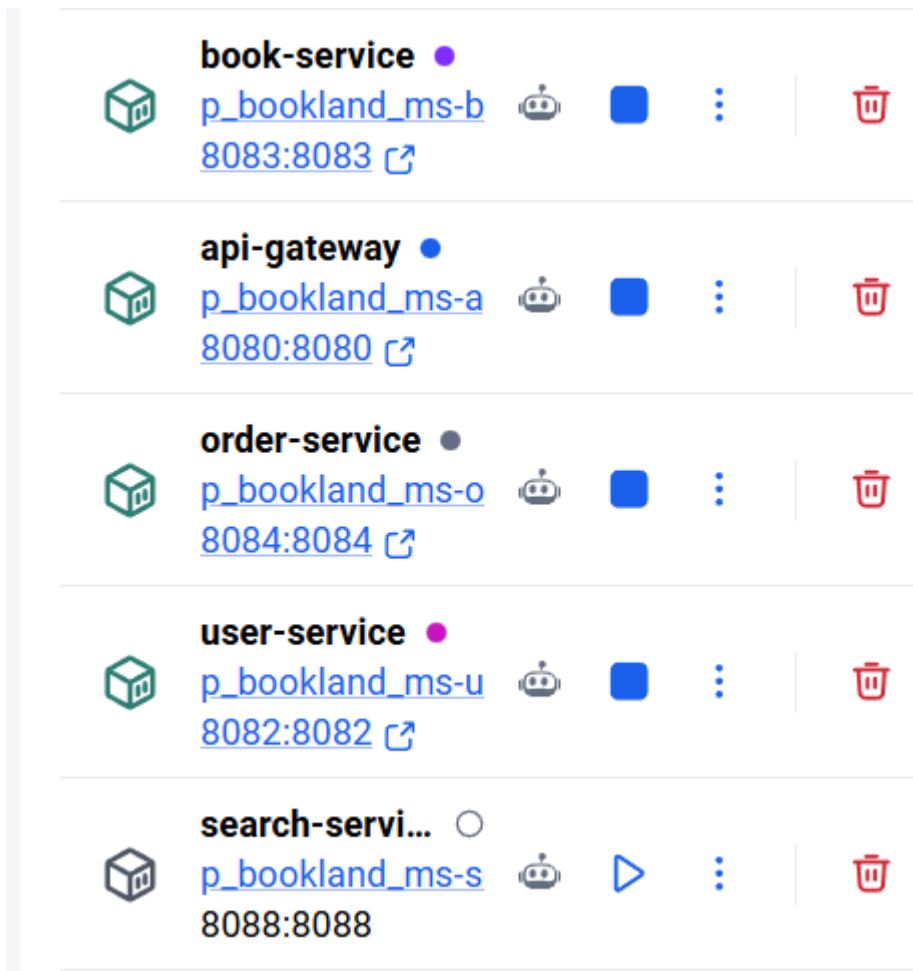


Hình 4.12 Kịch bản Tắt Search Service, Bật Book Service

4.2.5 Kiểm thử tính độc lập service

Tính độc lập của service được kiểm thử bằng cách dừng một service không nằm trên luồng nghiệp vụ chính và quan sát ảnh hưởng đến phần còn lại của hệ thống. Ví dụ, khi Search Service tạm thời dừng, chức năng tìm kiếm nâng cao có thể bị ảnh hưởng, nhưng Book Service vẫn có khả năng trả danh sách sách thông qua truy vấn database. Khi Notification Service gặp sự cố, luồng tạo đơn hàng vẫn có thể hoàn tất ở Order Service, trong khi phần gửi thông báo được ghi nhận lỗi và xử lý sau.

Kết quả kiểm thử cho thấy việc tách service và tách database giúp giảm phạm vi ảnh hưởng của lỗi. Một lỗi cục bộ không nhất thiết làm dừng toàn bộ hệ thống. Với các nghiệp vụ vẫn phụ thuộc đồng bộ vào service khác, ví dụ Order Service cần Book Service để kiểm tra thông tin sách, hệ thống hiện dựa vào xử lý ngoại lệ, ghi log và fallback cục bộ; circuit breaker/retry policy hoàn chỉnh là phần nên tiếp tục bổ sung để giới hạn lỗi lan truyền, giảm thời gian chờ và tăng khả năng phục hồi khi service phía sau chỉ lỗi tạm thời.



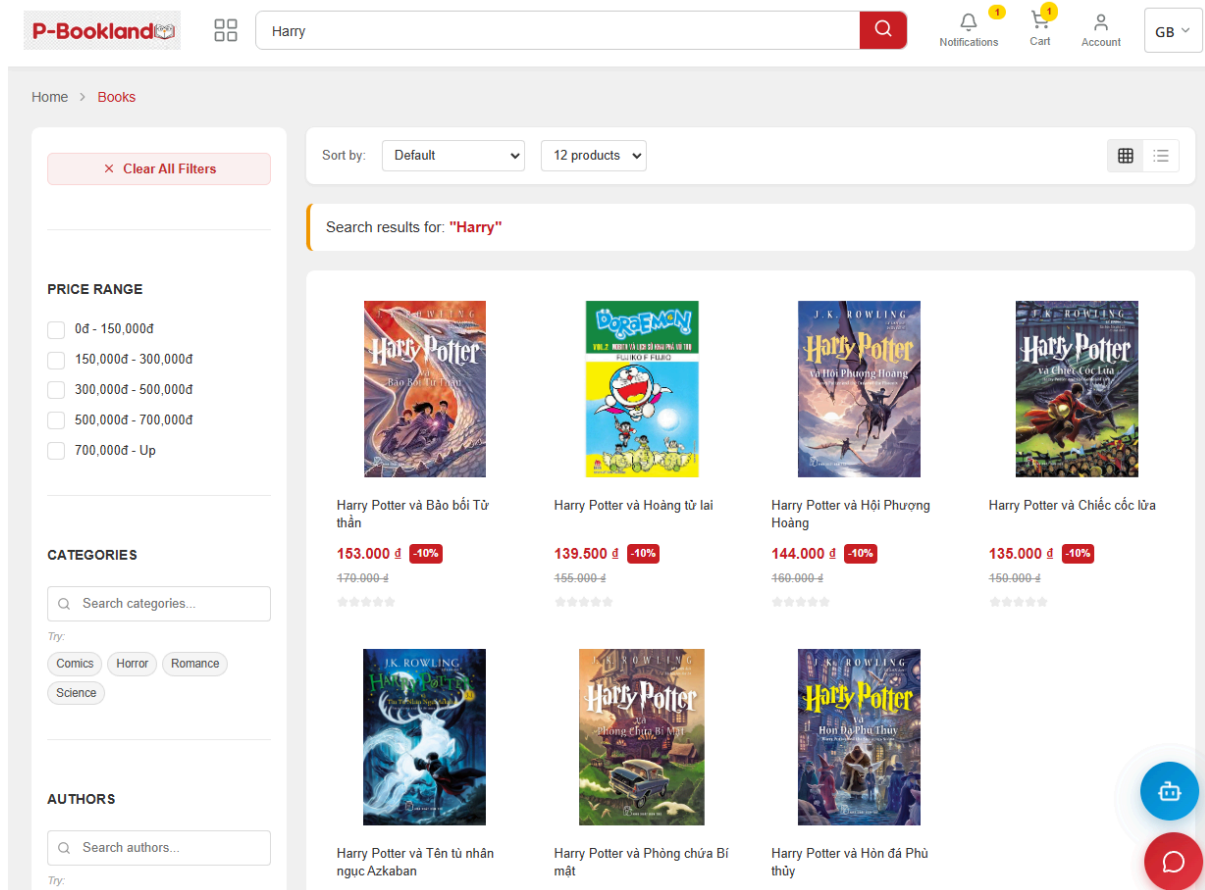
Hình 4.15 Kịch bản Tắt Search Service, Bật Book Service

```

2026-05-26T22:46:29.769Z INFO 1 --- [book-service] [nio-8083-exec-4] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:46:29.790Z INFO 1 --- [book-service] [nio-8083-exec-6] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:49:43.600Z INFO 1 --- [book-service] [nio-8083-exec-8] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:49:51.524Z INFO 1 --- [book-service] [nio-8083-exec-5] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:49:58.420Z INFO 1 --- [book-service] [nio-8083-exec-9] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:50:03.772Z INFO 1 --- [book-service] [nio-8083-exec-2] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:56:04.564Z INFO 1 --- [book-service] [nio-8083-exec-3] com.bookland.book.service.BookService : Delegating book search to Elasticsearch via
search-service Feign Client...
2026-05-26T22:56:08.710Z ERROR 1 --- [book-service] [nio-8083-exec-3] com.bookland.book.service.BookService : Failed to search books via Elasticsearch,
falling back to Database Specification query.

```

Hình 4.16 Kết quả Log



Hình 4.17 Kết quả web vẫn hoạt động

4.3 Đánh giá kết quả

Qua quá trình triển khai và thử nghiệm, hệ thống BookLand đã đáp ứng các yêu cầu chính của đề tài. Hệ thống có nhiều microservice hoạt động độc lập, có RESTful API cho giao tiếp đồng bộ và có Kafka cho giao tiếp bất đồng bộ. Các service được đóng gói bằng Docker, có thể khởi chạy cùng hạ tầng phụ trợ bằng Docker Compose và có manifest Kubernetes phục vụ triển khai trong môi trường container orchestration.

Về chức năng, hệ thống đã hỗ trợ các nghiệp vụ cốt lõi của một website thương mại điện tử sách, bao gồm xác thực người dùng, quản lý sách, tìm kiếm, giỏ hàng, đặt hàng, khuyến mãi, thông báo, upload file và quản trị. Về kiến trúc, hệ thống thể hiện được các nguyên tắc quan trọng của microservice như tách miền nghiệp vụ, tách dữ liệu, giao tiếp qua API, giao tiếp qua message queue và cô lập lỗi ở mức cơ bản.

Về hạn chế, hệ thống vẫn cần bổ sung thêm các cơ chế production-grade như distributed tracing, circuit breaker/retry policy hoàn chỉnh, dead-letter queue, monitoring metrics/alerting, autoscaling và Saga/compensating transaction cho các nghiệp vụ nhiều bước. Những điểm này không làm thay đổi kết quả cốt lõi của đồ án, nhưng là các hướng cải tiến cần thiết để hệ thống có thể vận hành ổn định hơn trong môi trường thực tế.

4.4 Hạn chế và hướng phát triển

4.4.1 Hạn chế

Hệ thống chưa triển khai distributed tracing đầy đủ bằng OpenTelemetry, Zipkin, Jaeger hoặc Grafana Tempo. Vì vậy, khi một request đi qua nhiều service và phát sinh lỗi, việc liên kết toàn bộ hành trình request vẫn chủ yếu dựa vào thời điểm log, tên service và nội dung log tập trung trên Kibana.

Centralized logging đã được triển khai bằng EFK Stack trên Docker Compose và Kubernetes. Tuy nhiên, phần quan sát vận hành vẫn có thể tiếp tục nâng cấp bằng dashboard metrics, alert rule, correlation id/trace id xuyên service và biểu đồ tracing chuyên sâu.

Cơ chế retry và dead-letter queue cho Kafka chưa được cấu hình đầy đủ. Notification Consumer đã có bắt lỗi và ghi log, nhưng các message thất bại chưa được chuyển sang một topic lỗi riêng để xử lý lại có kiểm soát.

Cơ chế circuit breaker/retry policy chưa được triển khai hoàn chỉnh trong mã nguồn hiện tại. API Gateway đã có dependency/cấu hình nền Resilience4j, nhưng chưa gắn đầy đủ CircuitBreaker filter, fallback response và retry policy cho toàn bộ route hoặc Feign Client.

Luồng đặt hàng chưa áp dụng Saga Pattern đầy đủ. Khi một nghiệp vụ trải qua nhiều bước như kiểm tra sách, cập nhật tồn kho, tạo hóa đơn, thanh toán và gửi thông báo, hệ thống cần thêm cơ chế compensating transaction để bảo đảm tính nhất quán cuối cùng trong trường hợp một bước trung gian thất bại.

4.4.2 Hướng phát triển

Trong các phiên bản tiếp theo, hệ thống có thể bổ sung distributed tracing bằng OpenTelemetry kết hợp Jaeger, Zipkin hoặc Grafana Tempo để theo dõi request xuyên suốt nhiều service một cách trực quan hơn. Điều này giúp việc phát hiện lỗi và tối ưu hiệu năng trở nên chính xác hơn.

Hệ thống nên tiếp tục hoàn thiện lớp quan sát bằng cách bổ sung monitoring metrics/alerting bên cạnh EFK Stack hiện có. Khi đó, nhóm phát triển có thể theo dõi độ trễ, tỉ lệ lỗi và trạng thái tài nguyên theo thời gian thực.

Đối với Kafka, hệ thống nên bổ sung retry topic, dead-letter topic và cơ chế xử lý lại message thất bại. Cách tiếp cận này giúp tăng độ tin cậy cho các luồng bất đồng bộ như gửi email, thông báo realtime và đồng bộ dữ liệu tìm kiếm.

Đối với giao tiếp đồng bộ giữa các service, hệ thống nên bổ sung đầy đủ timeout, retry policy và circuit breaker theo từng nhóm API. Các cơ chế này giúp giới hạn tác động khi một service phía sau phản hồi chậm hoặc tạm thời không khả dụng.

Về triển khai, hệ thống có thể tiếp tục hoàn thiện Kubernetes bằng cách bổ sung Horizontal Pod Autoscaler, health check chi tiết, resource limit phù hợp và dashboard giám sát bằng Prometheus/Grafana. Đây là bước cần thiết để hệ thống có thể vận hành ổn định trong môi trường có tải biến động.

KẾT LUẬN

Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai, nhóm đã xây dựng được hệ thống BookLand theo định hướng kiến trúc microservice, đáp ứng các yêu cầu chính của đề tài môn Các hệ thống phân tán. Hệ thống đã được phân rã thành nhiều service độc lập, mỗi service đảm nhiệm một nhóm chức năng riêng như xác thực, quản lý người dùng, quản lý sách, xử lý đơn hàng, quản lý sự kiện, gửi thông báo, lưu trữ file và tìm kiếm. Việc phân tách này giúp hệ thống thể hiện rõ các đặc trưng của kiến trúc phân tán, bao gồm tính mô-đun, khả năng triển khai độc lập, khả năng mở rộng theo từng thành phần và khả năng cô lập lỗi ở mức cơ bản.

Về mặt giao tiếp, hệ thống đã kết hợp cả hai cơ chế đồng bộ và bất đồng bộ. RESTful API được sử dụng cho các luồng cần phản hồi trực tiếp như đăng nhập, truy vấn sách, xử lý giỏ hàng và tạo đơn hàng. Apache Kafka được sử dụng trong luồng gửi thông báo, giúp tách Order Service khỏi Notification Service và giảm sự phụ thuộc trực tiếp giữa các service. Bên cạnh đó, hệ thống còn sử dụng Elasticsearch để hỗ trợ tìm kiếm sách, MongoDB để lưu trữ thông báo, MySQL cho các dữ liệu nghiệp vụ quan hệ, Redis cho các tác vụ hỗ trợ và Docker Compose để triển khai môi trường chạy cục bộ.

Kết quả thử nghiệm cho thấy hệ thống đã thực hiện được các luồng nghiệp vụ chính của một nền tảng thương mại điện tử sách, bao gồm xác thực người dùng, quản lý sách, tìm kiếm, quản lý giỏ hàng, đặt hàng và gửi thông báo. Các service có thể chạy độc lập trong container và phối hợp với nhau thông qua API Gateway, RESTful API và Kafka. Ngoài ra, hệ thống cũng đã có các cơ chế xử lý lỗi cơ bản như lớp xử lý ngoại lệ tập trung, ghi log lỗi, bắt lỗi khi gọi service khác và fallback trong một số trường hợp như tìm kiếm.

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Centralized logging đã được triển khai bằng EFK Stack, nhưng distributed tracing, circuit breaker/retry policy hoàn chỉnh, dashboard metrics và cảnh báo vận hành vẫn cần tiếp tục bổ sung. Dead-letter queue cho Kafka và luồng đặt hàng theo Saga Pattern/compensating transaction cũng

cần hoàn thiện để xử lý nhất quán dữ liệu trong các tình huống lỗi phức tạp. Đây là những điểm cần tiếp tục nghiên cứu và phát triển nếu hệ thống được mở rộng theo hướng production.

Nhìn chung, đề tài đã giúp nhóm củng cố kiến thức về hệ thống phân tán và hiểu rõ hơn quá trình xây dựng một ứng dụng microservice từ thiết kế kiến trúc đến triển khai thực tế. BookLand không chỉ là một hệ thống thương mại điện tử sách ở mức đồ án, mà còn là cơ sở để nhóm tiếp tục phát triển các kỹ thuật nâng cao như giám sát tập trung, tự động mở rộng, xử lý lỗi nâng cao và tối ưu hiệu năng trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Martin Fowler, James Lewis. "Microservices".
<https://martinfowler.com/articles/microservices.html>, 2014.
- [2] Sam Newman. "Building Microservices: Designing Fine-Grained Systems". O'Reilly Media, 2015.
- [3] Spring Cloud Gateway Documentation.
<https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/>
- [4] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [5] Resilience4j Documentation. <https://resilience4j.readme.io/docs/circuitbreaker>
- [6] Elasticsearch Reference 8.x.
<https://www.elastic.co/guide/en/elasticsearch/reference/current/>
- [7] MinIO Documentation. <https://min.io/docs/minio/linux/index.html>
- [8] Docker Compose Reference. <https://docs.docker.com/compose/>
- [9] Kubernetes Documentation. <https://kubernetes.io/docs/home/>
- [10] VNPay Integration Guide. <https://sandbox.vnpayment.vn/apis/docs/>
- [11] SpringDoc OpenAPI 3 Documentation. <https://springdoc.org/>
- [12] Chris Richardson. "Microservices Patterns". Manning Publications, 2018.

PHỤ LỤC DỰ ÁN

1. Mã nguồn dự án:

[Yamaaaaaaaa/P_BookLand_MS at dev](https://github.com/Yamaaaaaaaa/P_BookLand_MS)

2. Production URL:

<https://p-bookland.sonasked1.workers.dev/>

3. Sơ đồ các luồng hoạt động:

- Luồng hóa đơn + thanh toán (Order Checkout Process Flow.svg)
- Luồng đồng bộ Message (Book Data Sync.svg)
- Luồng giao tiếp - chat (WebSocket Chat Service Flow.svg)

 Sơ đồ các luồng hoạt động